

```
/* 文件: miniprogram01/.eslintrc.js */
0001: module.exports = {
0002:   root: true,
0003:   env: {
0004:     browser: true,
0005:     node: true,
0006:     es2021: true
0007:   },
0008:   extends: ['eslint:recommended', 'plugin:vue/vue3-recommended', 'plugin:prettier/recommended'],
0009:   parserOptions: {
0010:     ecmaVersion: 'latest',
0011:     sourceType: 'module'
0012:   },
0013:   rules: {
0014:     'vue/multi-word-component-names': 'off'
0015:   }
0016: }
```

```
/* 文件: miniprogram01/.prettierrc.json */
0001: {
0002:   "semi": false,
0003:   "singleQuote": true,
0004:   "printWidth": 100,
0005:   "tabWidth": 2,
0006:   "trailingComma": "none"
0007: }
```

```
/* 文件: miniprogram01/cloudfunctions/volunteer-service/index.js */
0001: const cloud = require('wx-server-sdk')
0002:
0003: cloud.init({
0004:   env: cloud.DYNAMIC_CURRENT_ENV
0005: })
0006:
0007: const db = cloud.database()
0008: const _ = db.command
0009: const $ = db.command.aggregate
0010:
0011: const DEFAULT_PAGE_SIZE = 10
0012: const MAX_PAGE_SIZE = 50
0013: const MAX_CHECKIN_PHOTOS = 9
0014: const ADMIN_SESSION_TTL_MS = 7 * 24 * 60 * 60 * 1000
0015: const ROLE_SUPER_ADMIN = 'super-admin'
0016: const ROLE_ADMIN = 'admin'
0017: const ROLE_MEMBER = 'member'
0018:
0019: const HONOR_LEVEL_POINTS_MAP = {
0020:   national: 20,
0021:   provincial: 16,
0022:   bureau: 12,
0023:   factory: 10
0024: }
0025: const VOLUNTEER_MODULE_RULES = {
0026:   'red-culture': { min: 3, max: 10 },
0027:   'community-governance': { min: 1, max: 5 },
```

```
0028: 'enterprise-service': { min: 3, max: 10 },
0029: 'elder-help': { min: 1, max: 5 },
0030: 'other-service': { min: 1, max: 5 }
0031: }
0032:
0033: const ROUTE_ACTION_RULES = [
0034: { method: 'POST', route: '/auth/login', action: 'wechatLogin' },
0035: { method: 'POST', route: '/user/realname', action: 'bindUser' },
0036: { method: 'GET', route: '/user/profile', action: 'getUserProfile' },
0037: { method: 'POST', route: '/volunteer/submit', action: 'submitVolunteerDeclaration' },
0038: { method: 'GET', route: '/volunteer/records', action: 'getVolunteerRecords' },
0039: { method: 'POST', route: '/honor/submit', action: 'submitHonor' },
0040: { method: 'GET', route: '/honor/records', action: 'getHonorRecords' },
0041: { method: 'POST', route: '/admin/import', action: 'adminImport' },
0042: { method: 'GET', route: '/admin/dashboard', action: 'adminDashboardSummary' },
0043: { method: 'POST', route: '/admin/audit', action: 'adminAuditOperate' },
0044: { method: 'GET', route: '/admin/audit', action: 'adminAuditList' },
0045: { method: 'GET', route: '/admin/export', action: 'adminExport' },
0046: { method: 'GET', route: '/admin/users', action: 'adminGetUsers' },
0047: { method: 'POST', route: '/admin/users/role', action: 'adminSetUserRole' },
0048: { method: 'POST', route: '/admin/users/disable', action: 'adminDisableUser' }
0049: ]
0050:
0051: /** 褰捩鐫鎶栁鑼嶽辨硲寰冩紮緱煎岀泗戝嚙?HTTP 涓�岀殑绫?URL 鐔兼兼鐔兼煎紡鎶�?*/
0052: function normalizeRoutePath(rawPath = '') {
0053:   const cleanPath = String(rawPath || '')
0054:     .trim()
0055:     .split('?')[0]
0056:     .replace(/\\/g, '/')
0057:     .replace(/\/{2,}/g, '/')
0058:
0059:   if (!cleanPath) return ''
0060:
0061:   const normalized = cleanPath.startsWith('/') ? cleanPath : `/${cleanPath}`
0062:   return normalized.endsWith('/') && normalized.length > 1 ? normalized.slice(0, -1) : normalized
0063: }
0064:
0065: /** 淪攸爻璁 f 灑 HTTP body鑄岒敲鑼佸岀彨璁 穿 JSON 瀛樺岀涓?*/
0066: function parseBodyData(body) {
0067:   if (!body) return {}
0068:   if (typeof body === 'object') return body
0069:   if (typeof body !== 'string') return {}
0070:
0071:   try {
0072:     return JSON.parse(body)
0073:   } catch (err) {
0074:     return {}
0075:   }
0076: }
0077:
0078: /** 鐔兼煎岀 method + path 璁 f 灑涓�岀殑?action鑄岒敲鑼佸�寰�寰�?鐔兼煎岀?*/
0079: function resolveRouteAction(method, path) {
0080:   if (!path) return ''
0081:   for (const rule of ROUTE_ACTION_RULES) {
0082:     const methodMatch = rule.method === method
```

```
0083:   const pathMatch = path === rule.route || path.endsWith(rule.route)
0084:   if (methodMatch && pathMatch) {
0085:     return rule.action
0086:   }
0087: }
0088: return ""
0089: }
0090:
0091: /** 鐳煎 action 璋冩澶?REST 璁?璋冩澶?鍑?鐳澶?寰?鏋? action + data鎶?*/
0092: function resolveIncomingRequest(event = {}) {
0093:   const method = String(
0094:     event.httpMethod ||
0095:     event.method ||
0096:     event.requestContext?.httpMethod ||
0097:     event.requestContext?.http?.method ||
0098:     'POST'
0099:   )
0100:   .trim()
0101:   .toUpperCase()
0102:
0103:   const routePath = normalizeRoutePath(event.path || event.url || event.route)
0104:   const queryData = Object.assign({}, event.queryStringParameters || {}, event.query || {})
0105:   const bodyData = parseBodyData(event.body)
0106:   const eventData = event && typeof event.data === 'object' ? event.data : {}
0107:   const mergedData = Object.assign({}, queryData, bodyData, eventData)
0108:
0109:   const rawAction = String(event.action || "").trim()
0110:   const actionFromRouteText = rawAction.startsWith('/')
0111:     ? resolveRouteAction(method, normalizeRoutePath(rawAction))
0112:     : ""
0113:   const actionFromRoute = resolveRouteAction(method, routePath)
0114:   const action = actionFromRouteText || rawAction || actionFromRoute
0115:
0116:   return {
0117:     action,
0118:     method,
0119:     routePath,
0120:     data: mergedData
0121:   }
0122: }
0123:
0124: exports.main = async (event = {}) => {
0125:   const { action, data, method, routePath } = resolveIncomingRequest(event)
0126:   const { OPENID } = cloud.getWXContext()
0127:   const effectiveOpenid = String(OPENID || data.openid || data._openid || "").trim()
0128:
0129:   try {
0130:     switch (action) {
0131:       case 'wechatLogin':
0132:         return await wechatLogin(data, effectiveOpenid)
0133:       case 'adminLogin':
0134:         return await adminLogin(data, effectiveOpenid)
0135:       case 'bindUser':
0136:         return await bindUser(data, effectiveOpenid)
0137:       case 'getUserProfile':
```

```
0138:     return await getUserProfile(effectiveOpenid)
0139:
0140:   case 'getActivities':
0141:     return await getActivities(data, effectiveOpenid)
0142:   case 'getActivityById':
0143:     return await getActivityById(data.id, effectiveOpenid)
0144:   case 'publishActivity':
0145:     return await publishActivity(data, effectiveOpenid)
0146:
0147:   case 'signup':
0148:     return await signup(data.activityId, effectiveOpenid)
0149:   case 'cancelSignup':
0150:     return await cancelSignup(data.activityId, effectiveOpenid)
0151:   case 'getMySignups':
0152:     return await getMySignups(effectiveOpenid)
0153:
0154:   case 'submitCheckin':
0155:     return await submitCheckin(data, effectiveOpenid)
0156:   case 'getMyRecords':
0157:     return await getMyRecords(data, effectiveOpenid)
0158:   case 'submitVolunteerDeclaration':
0159:     return await submitVolunteerDeclaration(data, effectiveOpenid)
0160:   case 'getVolunteerRecords':
0161:     return await getVolunteerRecords(data, effectiveOpenid)
0162:
0163:   case 'getStatistics':
0164:     return await getStatistics(effectiveOpenid)
0165:   case 'exportReport':
0166:     return await exportReport(data, effectiveOpenid)
0167:
0168:   case 'submitHonor':
0169:     return await submitHonor(data, effectiveOpenid)
0170:   case 'getHonorRecords':
0171:     return await getHonorRecords(data, effectiveOpenid)
0172:
0173:   case 'adminGetUsers':
0174:     return await adminGetUsers(data, effectiveOpenid)
0175:   case 'adminGetUser':
0176:     return await adminGetUser(data, effectiveOpenid)
0177:   case 'getPointsLogs':
0178:     return await getPointsLogs(data, effectiveOpenid)
0179:   case 'adjustUserPoints':
0180:     return await adjustUserPoints(data, effectiveOpenid)
0181:   case 'adminGetCheckins':
0182:     return await adminGetCheckins(data, effectiveOpenid)
0183:   case 'auditCheckin':
0184:     return await auditCheckin(data, effectiveOpenid)
0185:   case 'adminGetStats':
0186:     return await adminGetStats(data, effectiveOpenid)
0187:   case 'adminGetHonors':
0188:     return await adminGetHonors(data, effectiveOpenid)
0189:   case 'adminAuditHonor':
0190:     return await adminAuditHonor(data, effectiveOpenid)
0191:   case 'adminImport':
0192:     return await adminImport(data, effectiveOpenid)
```

```
0193:   case 'adminDashboardSummary':
0194:     return await adminDashboardSummary(data, effectiveOpenid)
0195:   case 'adminAuditOperate':
0196:     return await adminAuditOperate(data, effectiveOpenid)
0197:   case 'adminAuditList':
0198:     return await adminAuditList(data, effectiveOpenid)
0199:   case 'adminExport':
0200:     return await adminExport(data, effectiveOpenid)
0201:   case 'adminSetUserRole':
0202:     return await adminSetUserRole(data, effectiveOpenid)
0203:   case 'adminDisableUser':
0204:     return await adminDisableUser(data, effectiveOpenid)
0205:
0206:   default:
0207:     return {
0208:       code: 400,
0209:       message: '未定义的业务动作',
0210:       detail: {
0211:         action: action || '',
0212:         method,
0213:         routePath
0214:       }
0215:     }
0216:   }
0217: } catch (err) {
0218:   console.error(`[Action ${action}] Error:`, err)
0219:   return { code: 500, message: err.message || '内部服务器错误' }
0220: }
0221: }
0222:
0223: function normalizePagination(page, pageSize) {
0224:   const safePage = Math.max(parseInt(page, 10) || 1, 1)
0225:   const safePageSize = Math.min(
0226:     Math.max(parseInt(pageSize, 10) || DEFAULT_PAGE_SIZE, 1),
0227:     MAX_PAGE_SIZE
0228:   )
0229:
0230:   return {
0231:     page: safePage,
0232:     pageSize: safePageSize,
0233:     skip: (safePage - 1) * safePageSize
0234:   }
0235: }
0236:
0237: function toBoundaryISO(input, endOfDay = false) {
0238:   if (!input) return ''
0239:
0240:   if (typeof input === 'string' && /^\d{4}-\d{2}-\d{2}$/.test(input)) {
0241:     return endOfDay ? `${input}T23:59:59.999Z` : `${input}T00:00:00.000Z`
0242:   }
0243:
0244:   const date = new Date(input)
0245:   if (Number.isNaN(date.getTime())) return ''
0246:
0247:   if (endOfDay) {
```

```
0248:   date.setHours(23, 59, 59, 999)
0249: } else {
0250:   date.setHours(0, 0, 0, 0)
0251: }
0252:
0253: return date.toISOString()
0254: }
0255:
0256: function resolveQueryWindow(params = {}) {
0257:   const { timeRange, startDate, endDate } = params
0258:
0259:   let startAt = toBoundaryISO(startDate, false)
0260:   let endAt = toBoundaryISO(endDate, true)
0261:
0262:   if (startAt || endAt) {
0263:     return { startAt, endAt }
0264:   }
0265:
0266:   const now = new Date()
0267:   const start = new Date(now)
0268:   const end = new Date(now)
0269:
0270:   if (timeRange === 'today') {
0271:     start.setHours(0, 0, 0, 0)
0272:     end.setHours(23, 59, 59, 999)
0273:     return { startAt: start.toISOString(), endAt: end.toISOString() }
0274:   }
0275:
0276:   if (timeRange === 'week') {
0277:     start.setHours(0, 0, 0, 0)
0278:     start.setDate(start.getDate() - 6)
0279:     end.setHours(23, 59, 59, 999)
0280:     return { startAt: start.toISOString(), endAt: end.toISOString() }
0281:   }
0282:
0283:   if (timeRange === 'month') {
0284:     start.setDate(1)
0285:     start.setHours(0, 0, 0, 0)
0286:     end.setHours(23, 59, 59, 999)
0287:     return { startAt: start.toISOString(), endAt: end.toISOString() }
0288:   }
0289:
0290:   return { startAt: '', endAt: '' }
0291: }
0292:
0293: async function safeRollback(transaction) {
0294:   try {
0295:     await transaction.rollback()
0296:   } catch (rollbackErr) {
0297:     console.warn('[transaction] rollback skipped:', rollbackErr && rollbackErr.message)
0298:   }
0299: }
0300:
0301: function chunkArray(values, size = 20) {
0302:   const chunks = []
```

```
0303: const list = Array.isArray(values) ? values : []
0304:
0305: for (let i = 0; i < list.length; i += size) {
0306:   chunks.push(list.slice(i, i + size))
0307: }
0308:
0309: return chunks
0310: }
0311:
0312: async function fetchByFieldIn(collectionName, field, values, extraWhere = {}) {
0313:   const uniqueValues = Array.from(new Set((values || []).filter(Boolean)))
0314:   if (uniqueValues.length === 0) {
0315:     return []
0316:   }
0317:
0318:   const chunks = chunkArray(uniqueValues, 20)
0319:   const result = []
0320:
0321:   for (const chunk of chunks) {
0322:     const whereQuery = Object.assign({}, extraWhere, {
0323:       [field]: _.in(chunk)
0324:     })
0325:
0326:     const res = await db.collection(collectionName).where(whereQuery).limit(100).get()
0327:
0328:     result.push(...(res.data || []))
0329:   }
0330:
0331:   return result
0332: }
0333:
0334: async function fetchAllByWhere(collectionName, whereQuery = {}, options = {}) {
0335:   const pageSize = Math.min(Math.max(Number(options.pageSize) || 100, 1), 100)
0336:   const orderByField = options.orderByField
0337:   const orderDirection = options.orderDirection || 'desc'
0338:   const result = []
0339:   let skip = 0
0340:
0341:   while (true) {
0342:     let query = db.collection(collectionName).where(whereQuery)
0343:
0344:     if (orderByField) {
0345:       query = query.orderBy(orderByField, orderDirection)
0346:     }
0347:
0348:     const res = await query.skip(skip).limit(pageSize).get()
0349:     const batch = res.data || []
0350:
0351:     result.push(...batch)
0352:
0353:     if (batch.length < pageSize) {
0354:       break
0355:     }
0356:
0357:     skip += batch.length
```

```
0358: }
0359:
0360: return result
0361: }
0362:
0363: /** 解析日期字符串, 解析失败时返回 null。 */
0364: function parseDateOrNull(value) {
0365:   if (!value) return null
0366:   const date = value instanceof Date ? value : new Date(value)
0367:   return Number.isNaN(date.getTime()) ? null : date
0368: }
0369:
0370: /** 将任意输入转为数组, 并过滤空值。 */
0371: function toArray(value) {
0372:   if (Array.isArray(value)) {
0373:     return value.filter(Boolean)
0374:   }
0375:   if (value == null || value === '') {
0376:     return []
0377:   }
0378:   return [value]
0379: }
0380:
0381: /** 归一化上传图片字段, 兼容 files/photos/proofs 等格式。 */
0382: function normalizePhotoList(value) {
0383:   if (Array.isArray(value)) {
0384:     return value
0385:       .map((item) => {
0386:         if (!item) return ''
0387:         if (typeof item === 'string') return item.trim()
0388:         if (typeof item === 'object')
0389:           return String(item.url || item.fileID || item.path || '').trim()
0390:         return ''
0391:       })
0392:       .filter(Boolean)
0393:       .slice(0, MAX_CHECKIN_PHOTOS)
0394:   }
0395:
0396:   if (typeof value === 'string') {
0397:     return value
0398:       .split(/[\n,, ; \s]+/g)
0399:       .map((item) => item.trim())
0400:       .filter(Boolean)
0401:       .slice(0, MAX_CHECKIN_PHOTOS)
0402:   }
0403:
0404:   return []
0405: }
0406:
0407: /** 解析年度筛选窗口, 返回起止 Date。 */
0408: function resolveYearWindow(yearValue) {
0409:   const year = Number(yearValue)
0410:   if (!Number.isInteger(year) || year < 1900 || year > 2999) {
0411:     return null
0412:   }
```



```
0413: const start = new Date(year, 0, 1, 0, 0, 0, 0)
0414: const end = new Date(year, 11, 31, 23, 59, 59, 999)
0415: return { start, end }
0416: }
0417:
0418: /** 映射审核状态文案与标签类型，供前端直接展示。 */
0419: function resolveStatusMeta(status) {
0420:   const map = {
0421:     pending: { statusText: '待审核', tagType: 'warning' },
0422:     approved: { statusText: '已通过', tagType: 'success' },
0423:     rejected: { statusText: '已驳回', tagType: 'error' }
0424:   }
0425:   return map[status] || map.pending
0426: }
0427:
0428: /** 读取对象中的第一个有效字段。 */
0429: function pickValue(source = {}, keys = []) {
0430:   for (const key of keys) {
0431:     const value = source[key]
0432:     if (value != null && String(value).trim() != '') {
0433:       return value
0434:     }
0435:   }
0436:   return ''
0437: }
0438:
0439: /** honor level 文本归一化为标准 id。 */
0440: function normalizeHonorLevel(level) {
0441:   const raw = String(level || '')
0442:     .trim()
0443:     .toLowerCase()
0444:   if (!raw) return ''
0445:   if (HONOR_LEVEL_POINTS_MAP[raw]) return raw
0446:
0447:   if (/国家/.test(raw)) return 'national'
0448:   if (/省|部/.test(raw)) return 'provincial'
0449:   if (/厅|局/.test(raw)) return 'bureau'
0450:   if (/厂|处|街道/.test(raw)) return 'factory'
0451:
0452:   return ''
0453: }
0454:
0455: /** volunteer 分类名称映射，兼容前端 moduleId。 */
0456: function resolveVolunteerCategory(moduleId = '', fallbackCategory = '') {
0457:   const moduleMap = {
0458:     'red-culture': '传承红色文化（关心下一代）',
0459:     'community-governance': '参与基层治理',
0460:     'enterprise-service': '服务企业发展',
0461:     'elder-help': '实施以老助老',
0462:     'other-service': '其他服务'
0463:   }
0464:   return moduleMap[moduleId] || fallbackCategory || '其他服务'
0465: }
0466:
0467: /** 前端志愿申报 payload 标准化。 */
```

```
0468: function normalizeVolunteerDeclarationPayload(data = {}) {
0469:   const moduleId = String(data.moduleId || data.activityCategory || '').trim()
0470:   const title = String(data.title || data.activityName || '').trim()
0471:   const location = String(data.location || data.activityLocation || '').trim()
0472:   const content = String(data.content || data.remark || '').trim()
0473:   const activityTimeRaw = pickValue(data, ['activityTime', 'checkedAt', 'time'])
0474:   const activityTime = parseDateOrNull(activityTimeRaw)
0475:   const declaredPoints = Number(pickValue(data, ['points', 'declaredPoints']))
0476:   const photos = normalizePhotoList(data.files || data.photos)
0477:   const serviceHours = Number(data.serviceHours)
0478:   const serviceCount = Number(data.serviceCount)
0479:
0480:   return {
0481:     moduleId,
0482:     title,
0483:     location,
0484:     content,
0485:     activityTime,
0486:     declaredPoints,
0487:     photos,
0488:     serviceHours,
0489:     serviceCount
0490:   }
0491: }
0492:
0493: /** 格式化时间为 YYYY-MM-DD, 兼容 Date/字符串。 */
0494: function formatYmd(dateValue) {
0495:   const date = parseDateOrNull(dateValue)
0496:   if (!date) return ''
0497:   const year = date.getFullYear()
0498:   const month = String(date.getMonth() + 1).padStart(2, '0')
0499:   const day = String(date.getDate()).padStart(2, '0')
0500:   return `${year}-${month}-${day}`
0501: }
0502:
0503: function isAdminSessionActive(user) {
0504:   if (!user || !user.adminSessionExpiresAt) return false
0505:
0506:   const expiresAt = new Date(user.adminSessionExpiresAt).getTime()
0507:   return Number.isFinite(expiresAt) && expiresAt > Date.now()
0508: }
0509:
0510: function normalizeRoleValue(role) {
0511:   const normalized = String(role || '').trim()
0512:   if (normalized === ROLE_SUPER_ADMIN || normalized === ROLE_ADMIN || normalized === ROLE_MEMBER) {
0513:     return normalized
0514:   }
0515:   return ROLE_MEMBER
0516: }
0517:
0518: function isAdminLikeRole(role) {
0519:   return role === ROLE_SUPER_ADMIN || role === ROLE_ADMIN
0520: }
0521:
0522: async function getUserRole(openid) {
```

```
0523: try {
0524:   const user = await getUserByOpenid(openid)
0525:   if (user) {
0526:     const role = normalizeRoleValue(user.role)
0527:     if (role === ROLE_SUPER_ADMIN) return ROLE_SUPER_ADMIN
0528:     if (role === ROLE_ADMIN || isAdminSessionActive(user)) return ROLE_ADMIN
0529:   }
0530: } catch (err) {
0531:   // 施儿酒臻溪缓 users 闊啁惜鏃踪公孥喙负鏊口€氩垚?   console.warn('[getUserRole] fallback to member:', err && err.message)
0532: }
0533:
0534: return ROLE_MEMBER
0535: }
0536:
0537: async function ensureAdmin(openid) {
0538:   const role = await getUserRole(openid)
0539:   if (!isAdminLikeRole(role)) {
0540:     return { code: 403, message: '涕呼鏊鐳悖鏊口墩琛炭口鏊嶄絃' }
0541:   }
0542:
0543:   return null
0544: }
0545:
0546: async function ensureSingleSuperAdmin() {
0547:   const countRes = await db.collection('users').where({ role: ROLE_SUPER_ADMIN }).count()
0548:   if (Number(countRes.total || 0) !== 1) {
0549:     return { code: 409, message: 'super-admin 配置异常，系统要求且仅允许 1 个 super-admin' }
0550:   }
0551:   return null
0552: }
0553:
0554: async function ensureSuperAdmin(openid) {
0555:   const singleError = await ensureSingleSuperAdmin()
0556:   if (singleError) return singleError
0557:
0558:   const user = await getUserByOpenid(openid)
0559:   if (!user || normalizeRoleValue(user.role) !== ROLE_SUPER_ADMIN) {
0560:     return { code: 403, message: '仅 super-admin 可执行该操作' }
0561:   }
0562:
0563:   return null
0564: }
0565:
0566: async function ensurePureAdmin(openid) {
0567:   const user = await getUserByOpenid(openid)
0568:   if (!user || normalizeRoleValue(user.role) !== ROLE_ADMIN) {
0569:     return { code: 403, message: '仅 admin 可执行该操作' }
0570:   }
0571:   return null
0572: }
0573:
0574: async function getActivities(params = {}, openid) {
0575:   const { page, pageSize, skip } = normalizePagination(params.page, params.pageSize)
0576:   const { keyword, location } = params
0577:   const { startAt, endAt } = resolveQueryWindow(params)
```

```
0578:
0579: const matchQuery = {}
0580:
0581: if (keyword) {
0582:   matchQuery.name = db.RegExp({ regexp: keyword, options: 'i' })
0583: }
0584:
0585: if (location) {
0586:   matchQuery.location = db.RegExp({ regexp: location, options: 'i' })
0587: }
0588:
0589: if (startAt) {
0590:   matchQuery.endTime = _gte(startAt)
0591: }
0592: if (endAt) {
0593:   matchQuery.startTime = _lte(endAt)
0594: }
0595:
0596: const countRes = await db.collection('activities').where(matchQuery).count()
0597:
0598: const activityRes = await db
0599:   .collection('activities')
0600:   .where(matchQuery)
0601:   .orderBy('createdAt', 'desc')
0602:   .skip(skip)
0603:   .limit(pageSize)
0604:   .get()
0605:
0606: const activities = activityRes.data || []
0607: let signupActivitySet = new Set()
0608:
0609: if (activities.length > 0) {
0610:   const signupList = await fetchByFieldIn(
0611:     'signups',
0612:     'activityId',
0613:     activities.map((item) => item._id),
0614:     { _openid: openid }
0615:   )
0616:
0617:   signupActivitySet = new Set((signupList || []).map((item) => item.activityId))
0618: }
0619:
0620: const list = activities.map((item) => ({
0621:   ...item,
0622:   isSignedUp: signupActivitySet.has(item._id)
0623: }))
0624:
0625: return {
0626:   code: 0,
0627:   data: {
0628:     list,
0629:     total: countRes.total,
0630:     page,
0631:     pageSize
0632:   }
}
```

```
0633: }
0634: }
0635:
0636: async function getActivityById(id, openid) {
0637:   if (!id) {
0638:     return { code: 400, message: '绯哄奸嫫海姘 ID' }
0639:   }
0640:
0641:   const activityRes = await db.collection('activities').where({ _id: id }).limit(1).get()
0642:   if (!activityRes.data || activityRes.data.length === 0) {
0643:     return { code: 404, message: '' }
0644:   }
0645:
0646:   const activity = activityRes.data[0]
0647:
0648:   const [signupRes, checkinRes] = await Promise.all([
0649:     db.collection('signups').where({ activityId: id, _openid: openid }).count(),
0650:     db.collection('records').where({ activityId: id, _openid: openid }).count()
0651:   ])
0652:
0653:   activity.isSignedUp = signupRes.total > 0
0654:   activity.isCheckedIn = checkinRes.total > 0
0655:
0656:   return { code: 0, data: activity }
0657: }
0658:
0659: async function publishActivity(form = {}, openid) {
0660:   const adminError = await ensureAdmin(openid)
0661:   if (adminError) return adminError
0662:
0663:   const name = String(form.name || '').trim()
0664:   const category = String(form.category || '').trim()
0665:   const location = String(form.location || '').trim()
0666:   const description = String(form.description || '').trim()
0667:   const startTime = String(form.startTime || '').trim()
0668:   const endTime = String(form.endTime || '').trim()
0669:   const maxCount = Number(form.maxCount)
0670:
0671:   if (!name || !location || !description || !startTime || !endTime) {
0672:     return { code: 400, message: '' }
0673:   }
0674:
0675:   if (!Number.isInteger(maxCount) || maxCount <= 0) {
0676:     return { code: 400, message: 'maxCount 蹇响』罇□□鐓存暱' }
0677:   }
0678:
0679:   const startTs = new Date(startTime).getTime()
0680:   const endTs = new Date(endTime).getTime()
0681:   if (!Number.isFinite(startTs) || !Number.isFinite(endTs) || startTs >= endTs) {
0682:     return { code: 400, message: '' }
0683:   }
0684:
0685:   const newActivity = {
0686:     name,
0687:     category,
```

```
0688:   location,
0689:   description,
0690:   startTime,
0691:   endTime,
0692:   maxCount,
0693:   publisherId: openid,
0694:   enrollCount: 0,
0695:   createdAt: db.serverDate(),
0696:   updatedAt: db.serverDate(),
0697:   status: 'recruiting'
0698: }
0699:
0700: const res = await db.collection('activities').add({ data: newActivity })
0701: return { code: 0, data: { _id: res._id, ...newActivity } }
0702: }
0703:
0704: async function signup(activityId, openid) {
0705:   if (!activityId) {
0706:     return { code: 400, message: '请提供活动ID' }
0707:   }
0708:
0709:   const transaction = await db.startTransaction()
0710:
0711:   try {
0712:     const activityRes = await transaction
0713:       .collection('activities')
0714:       .where({ _id: activityId })
0715:       .limit(1)
0716:       .get()
0717:     if (!activityRes.data || activityRes.data.length === 0) {
0718:       await safeRollback(transaction)
0719:       return { code: 404, message: '' }
0720:     }
0721:
0722:     const activity = activityRes.data[0]
0723:     const enrollCount = Number(activity.enrollCount || 0)
0724:     const maxCount = Number(activity.maxCount || 0)
0725:
0726:     if (activity.status === 'ended') {
0727:       await safeRollback(transaction)
0728:       return { code: 400, message: '活动已结束' }
0729:     }
0730:
0731:     if (maxCount > 0 && enrollCount >= maxCount) {
0732:       await safeRollback(transaction)
0733:       return { code: 400, message: '活动名额已满' }
0734:     }
0735:
0736:     const existing = await transaction
0737:       .collection('signups')
0738:       .where({
0739:         activityId,
0740:         _openid: openid
0741:       })
0742:       .count()
```

```
0743:
0744:   if (existing.total > 0) {
0745:     await safeRollback(transaction)
0746:     return { code: 400, message: '' }
0747:   }
0748:
0749:   await transaction.collection('signups').add({
0750:     data: {
0751:       activityId,
0752:       _openid: openid,
0753:       signupAt: db.serverDate()
0754:     }
0755:   })
0756:
0757:   await transaction
0758:     .collection('activities')
0759:     .doc(activityId)
0760:     .update({
0761:       data: { enrollCount: _inc(1) }
0762:     })
0763:
0764:   await transaction.commit()
0765:   return { code: 0, message: '签到成功' }
0766: } catch (err) {
0767:   await safeRollback(transaction)
0768:   console.error('[signup] error:', err)
0769:   return { code: 500, message: '签到失败' }
0770: }
0771: }
0772:
0773: async function cancelSignup(activityId, openid) {
0774:   if (!activityId) {
0775:     return { code: 400, message: '活动ID不存在' }
0776:   }
0777:
0778:   const transaction = await db.startTransaction()
0779:
0780:   try {
0781:     const signupCount = await transaction
0782:       .collection('signups')
0783:       .where({
0784:         activityId,
0785:         _openid: openid
0786:       })
0787:       .count()
0788:
0789:     if (signupCount.total === 0) {
0790:       await safeRollback(transaction)
0791:       return { code: 400, message: '' }
0792:     }
0793:
0794:     const removeRes = await transaction
0795:       .collection('signups')
0796:       .where({
0797:         activityId,
```

```
0798:   _openid: openid
0799: })
0800: .remove()
0801:
0802: const removed = Number(removeRes.removed || signupCount.total || 0)
0803: if (removed > 0) {
0804:   await transaction
0805:     .collection('activities')
0806:     .doc(activityId)
0807:     .update({
0808:       data: { enrollCount: _inc(-removed) }
0809:     })
0810: }
0811:
0812: await transaction.commit()
0813: return { code: 0, message: '' }
0814: } catch (err) {
0815:   await safeRollback(transaction)
0816:   console.error('[cancelSignup] error:', err)
0817:   return { code: 500, message: '菱欸桎鐳ㄣ悝漶辨触铤炭ㄣ绑整悛閱蔣瘕' }
0818: }
0819: }
0820:
0821: async function getMySignups(openid) {
0822:   const signups = await fetchAllByWhere(
0823:     'signups',
0824:     { _openid: openid },
0825:     { orderByField: 'signupAt', orderDirection: 'desc', pageSize: 100 }
0826:   )
0827:
0828:   if (!signups || signups.length === 0) {
0829:     return { code: 0, data: [] }
0830:   }
0831:
0832:   const activityIds = signups.map((item) => item.activityId).filter(Boolean)
0833:
0834:   const [activities, records] = await Promise.all([
0835:     fetchByFieldIn('activities', '_id', activityIds),
0836:     fetchByFieldIn('records', 'activityId', activityIds, { _openid: openid })
0837:   ])
0838:
0839:   const activityMap = new Map((activities || []).map((item) => [item._id, item]))
0840:   const checkedSet = new Set((records || []).map((item) => item.activityId))
0841:
0842:   const list = signups
0843:     .map((signup) => {
0844:       const activity = activityMap.get(signup.activityId)
0845:       if (!activity) return null
0846:
0847:       return {
0848:         ...activity,
0849:         activityId: signup.activityId,
0850:         signupId: signup._id,
0851:         signupAt: signup.signupAt,
0852:         isSignedUp: true,
```



```
0853:     isCheckedIn: checkedSet.has(signup.activityId)
0854:   }
0855: })
0856: .filter(Boolean)
0857:
0858: return { code: 0, data: list }
0859: }
0860:
0861: async function submitCheckin(data = {}, openid) {
0862:   const activityId = String(data.activityId || "").trim()
0863:   const declaredPoints = Number(data.declaredPoints)
0864:   const activityCategory = String(data.activityCategory || "").trim()
0865:   const serviceHours = Number(data.serviceHours)
0866:   const serviceCount = Number(data.serviceCount)
0867:   const photos = Array.isArray(data.photos) ? data.photos.filter(Boolean) : []
0868:   const remark = String(data.remark || "").trim()
0869:
0870:   if (!activityId) {
0871:     return { code: 400, message: '绯哄奸嫗海婢 ID' }
0872:   }
0873:
0874:   if (!Number.isFinite(declaredPoints) || declaredPoints <= 0) {
0875:     return { code: 400, message: '鑿虫姘纒口堵蹇响』涓烘口鑰存噓' }
0876:   }
0877:
0878:   if (photos.length > MAX_CHECKIN_PHOTOS) {
0879:     return { code: 400, message: `照片最多上传 ${MAX_CHECKIN_PHOTOS} 张` }
0880:   }
0881:
0882:   const transaction = await db.startTransaction()
0883:
0884:   try {
0885:     const signupRes = await transaction
0886:       .collection('signups')
0887:       .where({
0888:         activityId,
0889:         _openid: openid
0890:       })
0891:       .count()
0892:
0893:     if (signupRes.total === 0) {
0894:       await safeRollback(transaction)
0895:       return { code: 403, message: '' }
0896:     }
0897:
0898:     const existingRecordRes = await transaction
0899:       .collection('records')
0900:       .where({
0901:         activityId,
0902:         _openid: openid
0903:       })
0904:       .count()
0905:
0906:     if (existingRecordRes.total > 0) {
0907:       await safeRollback(transaction)
```

```
0908:   return { code: 400, message: '' }
0909: }
0910:
0911: const activityRes = await transaction
0912:   .collection('activities')
0913:   .where({ _id: activityId })
0914:   .limit(1)
0915:   .get()
0916: if (!activityRes.data || activityRes.data.length === 0) {
0917:   await safeRollback(transaction)
0918:   return { code: 404, message: '' }
0919: }
0920:
0921: const activity = activityRes.data[0]
0922:
0923: const record = {
0924:   activityId,
0925:   activityName: activity.name,
0926:   activityCategory: activity.category || activityCategory || '鐳朵糰鏈整姦',
0927:   activityLocation: activity.location,
0928:   declaredPoints,
0929:   photos,
0930:   remark,
0931:   _openid: openid,
0932:   checkedAt: db.serverDate(),
0933:   status: 'pending',
0934:   rejectReason: '',
0935:   updatedAt: db.serverDate()
0936: }
0937:
0938: if (Number.isFinite(serviceHours) && serviceHours > 0) {
0939:   record.serviceHours = serviceHours
0940: }
0941:
0942: if (Number.isInteger(serviceCount) && serviceCount > 0) {
0943:   record.serviceCount = serviceCount
0944: }
0945:
0946: const addRes = await transaction.collection('records').add({ data: record })
0947: await transaction.commit()
0948:
0949: return { code: 0, data: { _id: addRes._id, ...record } }
0950: } catch (err) {
0951:   await safeRollback(transaction)
0952:   console.error('[submitCheckin] error:', err)
0953:   return { code: 500, message: '鎖慣氮錄嶽嶺澶辨触銑崙口緋整悅閱蔣瘕' }
0954: }
0955: }
0956:
0957: async function getMyRecords(params = {}, openid) {
0958:   const { page, pageSize, skip } = normalizePagination(params.page, params.pageSize)
0959:
0960:   const countRes = await db.collection('records').where({ _openid: openid }).count()
0961:   const listRes = await db
0962:     .collection('records')
```

```
0963: .where({ _openid: openid })
0964: .orderBy('checkedAt', 'desc')
0965: .skip(skip)
0966: .limit(pageSize)
0967: .get()
0968:
0969: return {
0970:   code: 0,
0971:   data: {
0972:     list: listRes.data,
0973:     total: countRes.total,
0974:     page,
0975:     pageSize
0976:   }
0977: }
0978: }
0979:
0980: async function getStatistics(openid) {
0981:   const [userRes, checkinCountRes, honorCountRes] = await Promise.all([
0982:     db.collection('users').where({ _openid: openid }).limit(1).get(),
0983:     db.collection('records').where({ _openid: openid }).count(),
0984:     db.collection('honors').where({ _openid: openid }).count()
0985:   ])
0986:
0987:   const statsRes = await db
0988:     .collection('records')
0989:     .aggregate()
0990:     .match({
0991:       _openid: openid,
0992:       status: 'approved'
0993:     })
0994:     .group({
0995:       _id: null,
0996:       totalHours: $.sum('$serviceHours'),
0997:       totalCount: $.sum(1),
0998:       totalServed: $.sum('$serviceCount')
0999:     })
1000:     .end()
1001:
1002:   const byActivityRes = await db
1003:     .collection('records')
1004:     .aggregate()
1005:     .match({
1006:       _openid: openid,
1007:       status: 'approved'
1008:     })
1009:     .group({
1010:       _id: '$activityName',
1011:       personCount: $.sum('$serviceCount'),
1012:       totalHours: $.sum('$serviceHours')
1013:     })
1014:     .sort({ totalHours: -1 })
1015:     .end()
1016:
1017:   const byCategoryRes = await db
```

```
1018: .collection('records')
1019: .aggregate()
1020: .match({
1021:   _openid: openid,
1022:   status: 'approved'
1023: })
1024: .group({
1025:   _id: '$activityLocation',
1026:   count: $.sum(1),
1027:   totalHours: $.sum('$serviceHours')
1028: })
1029: .sort({ totalHours: -1 })
1030: .end()
1031:
1032: const stats = statsRes.list[0] || { totalHours: 0, totalCount: 0, totalServed: 0 }
1033: const user = userRes.data && userRes.data.length > 0 ? userRes.data[0] : null
1034:
1035: const [checkinRecordsRes, honorRecordsRes] = await Promise.all([
1036:   db
1037:     .collection('records')
1038:     .where({ _openid: openid })
1039:     .orderBy('checkedAt', 'desc')
1040:     .limit(50)
1041:     .get(),
1042:   db.collection('honors').where({ _openid: openid }).orderBy('createdAt', 'desc').limit(50).get()
1043: ])
1044:
1045: return {
1046:   code: 0,
1047:   data: {
1048:     totalHours: Number(stats.totalHours || 0),
1049:     totalCount: Number(stats.totalCount || 0),
1050:     totalServed: Number(stats.totalServed || 0),
1051:     totalPoints: Number(user?.totalPoints || 0),
1052:     totalCheckins: Number(checkinCountRes.total || 0),
1053:     totalHonors: Number(honorCountRes.total || 0),
1054:     checkinRecords: checkinRecordsRes.data || [],
1055:     honorRecords: honorRecordsRes.data || [],
1056:     byCategory: (byCategoryRes.list || []).map((item) => ({
1057:       category: item._id || '未分类',
1058:       count: Number(item.count || 0),
1059:       totalHours: Number(item.totalHours || 0)
1060:     })),
1061:     byActivity: (byActivityRes.list || []).map((item) => ({
1062:       activityName: item._id || '鏈口鍵媧海姩',
1063:       personCount: Number(item.personCount || 0),
1064:       totalHours: Number(item.totalHours || 0)
1065:     })))
1066:   }
1067: }
1068: }
1069:
1070: function escapeCsvCell(value) {
1071:   const text = String(value == null ? '' : value)
1072:   if (text.includes(',') || text.includes('"') || text.includes('\n')) {
```

```
1073:   return `${text.replace(/"/g, '"')}`
1074: }
1075:
1076: return text
1077: }
1078:
1079: function buildReportRows(stats) {
1080:   const rows = [
1081:     ['指标', '数值'],
1082:     ['总服务时长(小时)', stats.totalHours],
1083:     ['鏊備穿媧海姘僂°C瞢', stats.totalCount],
1084:     ['鏈整姦浜烘烘噓', stats.totalServed]
1085:   ]
1086:
1087:   if (Array.isArray(stats.byActivity) && stats.byActivity.length > 0) {
1088:     rows.push([])
1089:     rows.push(['媧海姘錫嘰O', '鏈整姦浜烘烘噓', '鏈整姦鏊嶅墮噓(灑忔祿)'])
1090:     stats.byActivity.forEach((item) => {
1091:       rows.push([item.activityName, item.personCount, item.totalHours])
1092:     })
1093:   }
1094:
1095:   return rows
1096: }
1097:
1098: function formatStamp(date) {
1099:   const yyyy = date.getFullYear()
1100:   const MM = String(date.getMonth() + 1).padStart(2, '0')
1101:   const dd = String(date.getDate()).padStart(2, '0')
1102:   const hh = String(date.getHours()).padStart(2, '0')
1103:   const mm = String(date.getMinutes()).padStart(2, '0')
1104:   const ss = String(date.getSeconds()).padStart(2, '0')
1105:   return `${yyyy}${MM}${dd}-${hh}${mm}${ss}`
1106: }
1107:
1108: async function exportReport(_params = {}, openid) {
1109:   const statsRes = await getStatistics(openid)
1110:   if (statsRes.code !== 0) {
1111:     return statsRes
1112:   }
1113:
1114:   const rows = buildReportRows(statsRes.data)
1115:   const csvBody = rows.map((row) => row.map(escapeCsvCell).join(',')).join('\n')
1116:
1117:   // UTF-8 BOM, 确保 Excel 打开中文不乱码
1118:   const csv = `\uFEFF${csvBody}`
1119:   const stamp = formatStamp(new Date())
1120:   const cloudPath = `volunteer-reports/${openid}/volunteer-report-${stamp}.csv`
1121:
1122:   const uploadRes = await cloud.uploadFile({
1123:     cloudPath,
1124:     fileContent: Buffer.from(csv, 'utf8')
1125:   })
1126:
1127:   return {
```

```
1128:   code: 0,
1129:   data: uploadRes.fileID
1130: }
1131: }
1132:
1133: function buildToken(openid) {
1134:   return `token_${openid}_${Date.now()}`
1135: }
1136:
1137: async function findAdminByCredential(account, password) {
1138:   const inputAccount = String(account || '').trim()
1139:   const inputPassword = String(password || '').trim()
1140:   if (!inputAccount || !inputPassword) return null
1141:
1142:   const res = await db
1143:     .collection('users')
1144:     .where({ role: _in([ROLE_ADMIN, ROLE_SUPER_ADMIN]) })
1145:     .limit(100)
1146:     .get()
1147:   const adminUsers = res.data || []
1148:
1149:   for (const user of adminUsers) {
1150:     const candidateAccounts = [
1151:       user.adminAccount,
1152:       user.account,
1153:       user.username,
1154:       user.loginAccount,
1155:       user.phone
1156:     ]
1157:     .map((v) => String(v == null ? '' : v).trim())
1158:     .filter(Boolean)
1159:
1160:     const candidatePasswords = [user.adminPassword, user.password, user.passwd, user.loginPassword]
1161:     .map((v) => String(v == null ? '' : v).trim())
1162:     .filter(Boolean)
1163:
1164:     if (candidateAccounts.includes(inputAccount) && candidatePasswords.includes(inputPassword)) {
1165:       return user
1166:     }
1167:   }
1168:
1169:   console.warn(
1170:     `[adminLogin] credential mismatch, account=${inputAccount}, adminCount=${adminUsers.length}`
1171:   )
1172:   return null
1173: }
1174:
1175: async function attachAdminSession(openid, account, adminUser) {
1176:   const user = await ensureUser(openid)
1177:   if (!user || !user._id) return null
1178:
1179:   const now = new Date()
1180:   const expiresAt = new Date(now.getTime() + ADMIN_SESSION_TTL_MS)
1181:
1182:   await db
```

```
1183: .collection('users')
1184: .doc(user_id)
1185: .update({
1186:   data: {
1187:     adminSessionAccount: account,
1188:     adminSessionUserId: adminUser?._id || "",
1189:     adminSessionAt: now,
1190:     adminSessionExpiresAt: expiresAt,
1191:     updatedAt: db.serverDate()
1192:   }
1193: })
1194:
1195: const latest = await db.collection('users').doc(user_id).get()
1196: return latest.data || user
1197: }
1198:
1199: async function adminLogin(data = {}, openid) {
1200:   const account = String(data.account || "").trim()
1201:   const password = String(data.password || "").trim()
1202:
1203:   if (!account || !password) {
1204:     return { code: 400, message: "" }
1205:   }
1206:   if (!openid) {
1207:     return { code: 400, message: '缙哄肝鑿儿垌鐳凶搞' }
1208:   }
1209:
1210:   const credentialUser = await findAdminByCredential(account, password)
1211:   if (!credentialUser) {
1212:     return { code: 401, message: "" }
1213:   }
1214:
1215:   await attachAdminSession(openid, account, credentialUser)
1216:   const normalizedUser = normalizeUserData(credentialUser)
1217:   const displayName = String(
1218:     credentialUser.nickName ||
1219:     credentialUser.nickname ||
1220:     credentialUser.realName ||
1221:     credentialUser.adminAccount ||
1222:     credentialUser.account ||
1223:     account
1224:   ).trim()
1225:   const userInfo = {
1226:     ...normalizedUser,
1227:     nickName: displayName,
1228:     nickname: displayName,
1229:     avatar: credentialUser.avatar || credentialUser.avatarUrl || "",
1230:     avatarUrl: credentialUser.avatarUrl || credentialUser.avatar || "",
1231:     role: normalizeRoleValue(normalizedUser?.role)
1232:   }
1233:
1234:   return {
1235:     code: 0,
1236:     data: {
1237:       token: buildToken(openid),
```

```
1238:   userInfo
1239: }
1240: }
1241: }
1242:
1243: function normalizeUserData(user) {
1244:   if (!user) return user
1245:   const rawRole = normalizeRoleValue(user.role)
1246:   const role =
1247:     rawRole === ROLE_SUPER_ADMIN
1248:       ? ROLE_SUPER_ADMIN
1249:       : rawRole === ROLE_ADMIN || isAdminSessionActive(user)
1250:         ? ROLE_ADMIN
1251:         : ROLE_MEMBER
1252:   return {
1253:     ...user,
1254:     totalPoints: Number(user.totalPoints || 0),
1255:     volunteerPoints: Number(user.volunteerPoints || 0),
1256:     honorPoints: Number(user.honorPoints || 0),
1257:     checkinCount: Number(user.checkinCount || 0),
1258:     role
1259:   }
1260: }
1261:
1262: async function getUserByOpenid(openid) {
1263:   if (!openid) return null
1264:   const res = await db.collection('users').where({ _openid: openid }).limit(1).get()
1265:   return res.data && res.data.length > 0 ? res.data[0] : null
1266: }
1267:
1268: /** 导入场景下按姓名 + 手机号匹配目标用户，不依赖模板传 openid 或身份证号。 */
1269: async function resolveImportTargetUser(row = {}) {
1270:   const explicitUserId = String(pickValue(row, ['userId', 'targetUserId', '用户ID])).trim()
1271:   if (explicitUserId) {
1272:     try {
1273:       const userRes = await db.collection('users').doc(explicitUserId).get()
1274:       if (userRes?.data?._id) {
1275:         return userRes.data
1276:       }
1277:     } catch (err) {
1278:       // no-op
1279:     }
1280:   }
1281:
1282:   const realName = String(pickValue(row, ['realName', 'userName', 'name', '用户姓名'])).trim()
1283:   const phone = String(pickValue(row, ['phone', 'mobile', '手机号', '手机号码'])).trim()
1284:
1285:   if (realName && phone) {
1286:     const userByNamePhoneRes = await db.collection('users').where({ realName, phone }).limit(2).get()
1287:     if ((userByNamePhoneRes.data || []).length === 1) {
1288:       return userByNamePhoneRes.data[0]
1289:     }
1290:   }
1291:
1292:   if (realName) {
```



```
1293:   const userByNameRes = await db.collection('users').where({ realName }).limit(2).get()
1294:   if ((userByNameRes.data || []).length === 1) {
1295:     return userByNameRes.data[0]
1296:   }
1297: }
1298:
1299: return null
1300: }
1301:
1302: async function ensureUser(openid) {
1303:   if (!openid) return null
1304:   const existing = await getUserByOpenid(openid)
1305:   if (existing) return existing
1306:
1307:   const data = {
1308:     _openid: openid,
1309:     realName: "",
1310:     phone: "",
1311:     role: 'member',
1312:     totalPoints: 0,
1313:     volunteerPoints: 0,
1314:     honorPoints: 0,
1315:     checkinCount: 0,
1316:     bindAt: null,
1317:     createdAt: db.serverDate(),
1318:     updatedAt: db.serverDate()
1319:   }
1320:
1321:   const res = await db.collection('users').add({ data })
1322:   const created = await db.collection('users').doc(res._id).get()
1323:   return created.data || { _id: res._id, ...data }
1324: }
1325:
1326: async function wechatLogin(_data, openid) {
1327:   if (!openid) {
1328:     return { code: 400, message: '请正确扫描小程序码' }
1329:   }
1330:
1331:   const user = await ensureUser(openid)
1332:   if (user?.isDeleted || String(user?.status || "") === 'disabled') {
1333:     return { code: 403, message: '账号已禁用，请联系管理员' }
1334:   }
1335:   const normalized = normalizeUserData(user)
1336:   const needBinding = !normalized?.realName || !normalized?.phone
1337:
1338:   return {
1339:     code: 0,
1340:     data: {
1341:       needBinding,
1342:       openid,
1343:       token: buildToken(openid),
1344:       userInfo: normalized
1345:     }
1346:   }
1347: }
```

```
1348:
1349: async function bindUser(data = {}, openidFromCtx) {
1350:   const openid = String(data.openid || openidFromCtx || "").trim()
1351:   const realName = String(data.realName || "").trim()
1352:   const phone = String(data.phone || "").trim()
1353:
1354:   if (!openid) {
1355:     return { code: 400, message: '绑哄肝 openid' }
1356:   }
1357:   if (!realName || !phone) {
1358:     return { code: 400, message: "" }
1359:   }
1360:
1361:   const user = await getUserByOpenid(openid)
1362:   if (user?.isDeleted || String(user?.status || "") === 'disabled') {
1363:     return { code: 403, message: '账号已禁用, 请联系管理员' }
1364:   }
1365:   const updateData = {
1366:     realName,
1367:     phone,
1368:     updatedAt: db.serverDate(),
1369:     bindAt: user?.bindAt || db.serverDate()
1370:   }
1371:
1372:   if (user && user._id) {
1373:     await db.collection('users').doc(user._id).update({ data: updateData })
1374:   } else {
1375:     await db.collection('users').add({
1376:       data: {
1377:         _openid: openid,
1378:         role: 'member',
1379:         totalPoints: 0,
1380:         volunteerPoints: 0,
1381:         honorPoints: 0,
1382:         checkinCount: 0,
1383:         createdAt: db.serverDate(),
1384:         ...updateData
1385:       }
1386:     })
1387:   }
1388:
1389:   const latest = await getUserByOpenid(openid)
1390:   return {
1391:     code: 0,
1392:     data: {
1393:       needBinding: false,
1394:       openid,
1395:       token: buildToken(openid),
1396:       userInfo: normalizeUserData(latest)
1397:     }
1398:   }
1399: }
1400:
1401: /** 获取用户资料与积分汇总, 匹配前端 /user/profile 接口。 */
1402: async function getUserProfile(openid) {
```

```
1403: if (!openid) {
1404:   return { code: 400, message: '缺少用户标识' }
1405: }
1406:
1407: let user = await ensureUser(openid)
1408: if (user?.isDeleted || String(user?.status || '') === 'disabled') {
1409:   return { code: 403, message: '账号已禁用, 请联系管理员' }
1410: }
1411: let normalizedUser = normalizeUserData(user)
1412: const storedVolunteerPoints = Number(normalizedUser?.volunteerPoints || 0)
1413: const storedHonorPoints = Number(normalizedUser?.honorPoints || 0)
1414: const storedTotalPoints = Number(normalizedUser?.totalPoints || 0)
1415: const needBinding = !normalizedUser?.realName || !normalizedUser?.phone
1416: const needMigrateScoreFields =
1417:   !Object.prototype.hasOwnProperty.call(user || {}, 'volunteerPoints') ||
1418:   !Object.prototype.hasOwnProperty.call(user || {}, 'honorPoints') ||
1419:   storedTotalPoints !== storedVolunteerPoints + storedHonorPoints
1420:
1421: if (needMigrateScoreFields) {
1422:   const [volunteerAgg, honorAgg] = await Promise.all([
1423:     db
1424:       .collection('records')
1425:       .aggregate()
1426:       .match({ _openid: openid, status: 'approved' })
1427:       .group({
1428:         _id: null,
1429:         volunteerPoints: $.sum('$declaredPoints')
1430:       })
1431:       .end(),
1432:     db
1433:       .collection('honors')
1434:       .aggregate()
1435:       .match({ _openid: openid, status: 'approved' })
1436:       .group({
1437:         _id: null,
1438:         honorPoints: $.sum('$honorPoints')
1439:       })
1440:       .end()
1441:   ])
1442:
1443:   const volunteerPoints = Number(volunteerAgg.list[0]?.volunteerPoints || 0)
1444:   const honorPoints = Number(honorAgg.list[0]?.honorPoints || 0)
1445:   const totalPoints = volunteerPoints + honorPoints
1446:
1447:   await db
1448:     .collection('users')
1449:     .doc(user._id)
1450:     .update({
1451:       data: {
1452:         volunteerPoints,
1453:         honorPoints,
1454:         totalPoints,
1455:         updatedAt: db.serverDate()
1456:       }
1457:     })
1458: }
```

```
1458:
1459:   user = await getUserByOpenid(openid)
1460:   normalizedUser = normalizeUserData(user)
1461: }
1462:
1463: const volunteerPoints = Number(normalizedUser?.volunteerPoints || 0)
1464: const honorPoints = Number(normalizedUser?.honorPoints || 0)
1465: const totalPoints = Number(normalizedUser?.totalPoints || volunteerPoints + honorPoints)
1466:
1467: return {
1468:   code: 0,
1469:   data: {
1470:     needBinding,
1471:     userInfo: normalizedUser,
1472:     scoreSummary: {
1473:       volunteerPoints,
1474:       honorPoints,
1475:       totalPoints
1476:     }
1477:   }
1478: }
1479: }
1480:
1481: /** 提交志愿服务申报，匹配前端 /volunteer/submit 接口。 */
1482: async function submitVolunteerDeclaration(data = {}, openid) {
1483:   if (!openid) {
1484:     return { code: 400, message: '缺少用户标识' }
1485:   }
1486:
1487:   const payload = normalizeVolunteerDeclarationPayload(data)
1488:   if (!payload.activityTime || !payload.location || !payload.title || !payload.content) {
1489:     return { code: 400, message: '请完整填写志愿申报信息' }
1490:   }
1491:   if (!Number.isFinite(payload.declaredPoints) || payload.declaredPoints <= 0) {
1492:     return { code: 400, message: '申报积分不合法' }
1493:   }
1494:   if (payload.photos.length === 0) {
1495:     return { code: 400, message: '请上传佐证材料' }
1496:   }
1497:   if (payload.photos.length > MAX_CHECKIN_PHOTOS) {
1498:     return { code: 400, message: `佐证材料最多上传 ${MAX_CHECKIN_PHOTOS} 张` }
1499:   }
1500:
1501:   if (payload.moduleId && VOLUNTEER_MODULE_RULES[payload.moduleId]) {
1502:     const { min, max } = VOLUNTEER_MODULE_RULES[payload.moduleId]
1503:     if (payload.declaredPoints < min || payload.declaredPoints > max) {
1504:       return { code: 400, message: `该模块积分范围为 ${min}-${max}` }
1505:     }
1506:   }
1507:
1508:   await ensureUser(openid)
1509:
1510:   const record = {
1511:     activityId: String(data.activityId || `manual-${Date.now()}`),
1512:     activityName: payload.title,
```

```
1513:   activityCategory: resolveVolunteerCategory(
1514:     payload.moduleId,
1515:     String(data.activityCategory || '').trim()
1516:   ),
1517:   activityLocation: payload.location,
1518:   declaredPoints: payload.declaredPoints,
1519:   photos: payload.photos,
1520:   remark: payload.content,
1521:   _openid: openid,
1522:   checkedAt: payload.activityTime,
1523:   status: 'pending',
1524:   rejectReason: '',
1525:   updatedAt: db.serverDate()
1526: }
1527:
1528: if (payload.moduleId) {
1529:   record.moduleId = payload.moduleId
1530: }
1531: if (Number.isFinite(payload.serviceHours) && payload.serviceHours > 0) {
1532:   record.serviceHours = payload.serviceHours
1533: }
1534: if (Number.isFinite(payload.serviceCount) && payload.serviceCount > 0) {
1535:   record.serviceCount = Math.floor(payload.serviceCount)
1536: }
1537:
1538: const addRes = await db.collection('records').add({ data: record })
1539: return {
1540:   code: 0,
1541:   data: {
1542:     id: addRes._id
1543:   }
1544: }
1545: }
1546:
1547: /** 查询志愿申报记录，匹配前端 /volunteer/records 接口。 */
1548: async function getVolunteerRecords(params = {}, openid) {
1549:   if (!openid) {
1550:     return { code: 400, message: '缺少用户标识' }
1551:   }
1552:
1553:   const { page, pageSize, skip } = normalizePagination(params.page, params.pageSize)
1554:   const status = String(params.status || '').trim()
1555:   const moduleId = String(params.moduleId || '').trim()
1556:   const yearWindow = resolveYearWindow(params.year)
1557:
1558:   const whereQuery = { _openid: openid }
1559:   if (status) {
1560:     whereQuery.status = status
1561:   }
1562:   if (yearWindow) {
1563:     whereQuery.checkedAt = _.gte(yearWindow.start).and(_.lte(yearWindow.end))
1564:   }
1565:
1566:   let rawList = []
1567:   let total = 0
```

```
1568: if (moduleId) {
1569:   const allList = await fetchAllByWhere('records', whereQuery, {
1570:     orderByField: 'checkedAt',
1571:     orderDirection: 'desc',
1572:     pageSize: 100
1573:   })
1574:   const filteredList = allList.filter(
1575:     (item) => item.moduleId === moduleId || item.activityCategory === moduleId
1576:   )
1577:   total = filteredList.length
1578:   rawList = filteredList.slice(skip, skip + pageSize)
1579: } else {
1580:   const [countRes, listRes] = await Promise.all([
1581:     db.collection('records').where(whereQuery).count(),
1582:     db
1583:       .collection('records')
1584:       .where(whereQuery)
1585:       .orderBy('checkedAt', 'desc')
1586:       .skip(skip)
1587:       .limit(pageSize)
1588:       .get()
1589:   ])
1590:   total = countRes.total
1591:   rawList = listRes.data || []
1592: }
1593:
1594: const list = rawList.map((item) => {
1595:   const statusMeta = resolveStatusMeta(item.status || 'pending')
1596:   const declaredPoints = Number(item.declaredPoints || 0)
1597:   return {
1598:     id: item._id,
1599:     type: 'volunteer',
1600:     moduleId: item.moduleId || '',
1601:     categoryName: item.activityCategory || resolveVolunteerCategory(item.moduleId || ''),
1602:     title: item.activityName || '',
1603:     activityTime: formatYmd(item.checkedAt),
1604:     submitTime: formatYmd(item.updatedAt || item.checkedAt),
1605:     location: item.activityLocation || '',
1606:     content: item.remark || '',
1607:     points: declaredPoints,
1608:     claimedPoints: declaredPoints,
1609:     approvedPoints: item.status === 'approved' ? declaredPoints : 0,
1610:     evidenceFiles: toArray(item.photos),
1611:     rejectReason: item.rejectReason || '',
1612:     status: item.status || 'pending',
1613:     statusText: statusMeta.statusText,
1614:     tagType: statusMeta.tagType
1615:   }
1616: })
1617:
1618: return {
1619:   code: 0,
1620:   data: {
1621:     list,
1622:     total,
```

```
0066: .page-shell--mine {
0067:   background:
0068:     radial-gradient(circle at top right, rgba(9, 157, 152, 0.18), transparent 24%),
0069:     radial-gradient(circle at top left, rgba(45, 119, 255, 0.18), transparent 30%),
0070:     linear-gradient(180deg, #f1fbff 0%, #eef9f8 42%, #f8fbff 100%);
0071: }
0072:
0073: .page-shell--admin {
0074:   background:
0075:     radial-gradient(circle at top right, rgba(67, 145, 255, 0.18), transparent 24%),
0076:     radial-gradient(circle at top left, rgba(22, 163, 74, 0.12), transparent 32%),
0077:     linear-gradient(180deg, #eef4ff 0%, #eef7fb 46%, #f8fbff 100%);
0078: }
0079:
0080: .page-shell--form::after {
0081:   bottom: 140px;
0082: }
0083:
0084: .page-hero {
0085:   position: relative;
0086:   overflow: hidden;
0087:   border-radius: 30px;
0088:   padding: 24px 22px;
0089:   color: #ffffff;
0090:   background: linear-gradient(135deg, #1d63d8 0%, #177ddc 48%, #16b39f 100%);
0091:   box-shadow: 0 22px 40px rgba(25, 88, 181, 0.22);
0092: }
0093:
0094: .page-hero::before,
0095: .page-hero::after {
0096:   content: "";
0097:   position: absolute;
0098:   pointer-events: none;
0099: }
0100:
0101: .page-hero::before {
0102:   top: -34px;
0103:   right: -26px;
0104:   width: 154px;
0105:   height: 154px;
0106:   border-radius: 50%;
0107:   background:
0108:     radial-gradient(circle at center, rgba(255, 255, 255, 0.46), rgba(255, 255, 255, 0.04) 62%, transparent 7...
0109: }
0110:
0111: .page-hero::after {
0112:   left: -10px;
0113:   bottom: -10px;
0114:   width: 180px;
0115:   height: 72px;
0116:   border-radius: 72px;
0117:   opacity: 0.34;
0118:   background:
0119:     repeating-linear-gradient(
0120:       90deg,
```

```
0121:   rgba(255, 255, 255, 0.24) 0 2px,
0122:   transparent 2px 16px
0123: );
0124: }
0125:
0126: .page-hero--honor {
0127:   background: linear-gradient(135deg, #1d63d8 0%, #327de0 42%, #e5a72f 100%);
0128: }
0129:
0130: .page-hero--mine {
0131:   background: linear-gradient(135deg, #0f5ccf 0%, #1d78d7 40%, #0fa89d 100%);
0132: }
0133:
0134: .page-hero--admin {
0135:   background: linear-gradient(135deg, #1648a5 0%, #2468c9 48%, #1b9b8f 100%);
0136: }
0137:
0138: .hero-badge {
0139:   display: inline-flex;
0140:   align-items: center;
0141:   gap: 8px;
0142:   min-height: 36px;
0143:   padding: 0 14px;
0144:   border-radius: 999px;
0145:   font-size: 14px;
0146:   font-weight: 700;
0147:   background: rgba(255, 255, 255, 0.18);
0148: }
0149:
0150: .hero-title {
0151:   display: block;
0152:   margin-top: 18px;
0153:   font-size: 24px;
0154:   font-weight: 800;
0155:   line-height: 1.45;
0156: }
0157:
0158: .hero-subtitle {
0159:   display: block;
0160:   margin-top: 10px;
0161:   font-size: 15px;
0162:   line-height: 1.7;
0163:   color: rgba(255, 255, 255, 0.92);
0164: }
0165:
0166: .hero-metrics {
0167:   display: flex;
0168:   flex-wrap: wrap;
0169:   gap: 12px;
0170:   margin-top: 18px;
0171: }
0172:
0173: .hero-pill {
0174:   min-width: 104px;
0175:   min-height: 56px;
```



```
0176: padding: 10px 14px;
0177: border-radius: 18px;
0178: background: rgba(255, 255, 255, 0.16);
0179: }
0180:
0181: .hero-pill__label {
0182:   display: block;
0183:   font-size: 13px;
0184:   color: rgba(255, 255, 255, 0.84);
0185: }
0186:
0187: .hero-pill__value {
0188:   display: block;
0189:   margin-top: 8px;
0190:   font-size: 24px;
0191:   font-weight: 800;
0192:   line-height: 1;
0193: }
0194:
0195: .section-heading {
0196:   display: flex;
0197:   align-items: center;
0198:   justify-content: space-between;
0199:   gap: 12px;
0200:   margin-top: 2px;
0201: }
0202:
0203: .section-heading__title {
0204:   font-size: 21px;
0205:   font-weight: 800;
0206:   color: var(--app-text-primary);
0207: }
0208:
0209: .section-heading__desc {
0210:   font-size: 14px;
0211:   color: var(--app-text-muted);
0212: }
0213:
0214: .themed-card,
0215: .glass-card,
0216: .data-card,
0217: .list-card {
0218:   position: relative;
0219:   overflow: hidden;
0220:   border-radius: 24px;
0221:   background: rgba(255, 255, 255, 0.94);
0222:   border: 1px solid rgba(172, 199, 231, 0.45);
0223:   box-shadow: 0 18px 34px rgba(19, 58, 107, 0.08);
0224: }
0225:
0226: .themed-card::before,
0227: .glass-card::before,
0228: .data-card::before,
0229: .list-card::before {
0230:   content: "";
```

```
0231: position: absolute;
0232: inset: 0 auto auto 0;
0233: width: 100%;
0234: height: 6px;
0235: background: linear-gradient(90deg, #1d63d8, #18a68f, #f0b53a);
0236: opacity: 0.88;
0237: }
0238:
0239: .themed-card,
0240: .glass-card,
0241: .list-card {
0242: padding: 18px;
0243: }
0244:
0245: .data-card {
0246: padding: 18px 18px 20px;
0247: }
0248:
0249: .card-kicker {
0250: display: inline-flex;
0251: align-items: center;
0252: gap: 6px;
0253: min-height: 30px;
0254: padding: 0 12px;
0255: border-radius: 999px;
0256: background: var(--app-fill-soft);
0257: color: var(--app-primary-deep);
0258: font-size: 13px;
0259: font-weight: 700;
0260: }
0261:
0262: .card-title-main {
0263: display: block;
0264: margin-top: 12px;
0265: font-size: 20px;
0266: font-weight: 800;
0267: color: var(--app-text-primary);
0268: line-height: 1.45;
0269: }
0270:
0271: .card-text-main {
0272: display: block;
0273: margin-top: 8px;
0274: font-size: 14px;
0275: line-height: 1.7;
0276: color: var(--app-text-secondary);
0277: }
0278:
0279: .feature-grid {
0280: display: grid;
0281: grid-template-columns: 1fr;
0282: gap: 14px;
0283: }
0284:
0285: .feature-card {
```

```
0286: position: relative;
0287: overflow: hidden;
0288: border-radius: 24px;
0289: padding: 20px 18px;
0290: min-height: 150px;
0291: display: flex;
0292: flex-direction: column;
0293: justify-content: space-between;
0294: color: #ffffff;
0295: box-shadow: 0 18px 34px rgba(26, 77, 148, 0.16);
0296: }
0297:
0298: .feature-card::before {
0299:   content: "";
0300:   position: absolute;
0301:   top: -20px;
0302:   right: -12px;
0303:   width: 118px;
0304:   height: 118px;
0305:   border-radius: 28px;
0306:   background: rgba(255, 255, 255, 0.16);
0307:   transform: rotate(18deg);
0308: }
0309:
0310: .feature-card::after {
0311:   content: "";
0312:   position: absolute;
0313:   left: 18px;
0314:   bottom: 14px;
0315:   width: 90px;
0316:   height: 90px;
0317:   border-radius: 50%;
0318:   border: 1px solid rgba(255, 255, 255, 0.18);
0319: }
0320:
0321: .feature-card--volunteer {
0322:   background: linear-gradient(135deg, #1d63d8 0%, #14a78b 100%);
0323: }
0324:
0325: .feature-card--honor {
0326:   background: linear-gradient(135deg, #1d63d8 0%, #e7a634 100%);
0327: }
0328:
0329: .feature-card--admin {
0330:   background: linear-gradient(135deg, #1648a5 0%, #2471cf 56%, #16a58f 100%);
0331: }
0332:
0333: .feature-card__head,
0334: .feature-card__foot {
0335:   position: relative;
0336:   z-index: 1;
0337: }
0338:
0339: .feature-card__icon {
0340:   width: 54px;
```

```
0341: height: 54px;
0342: border-radius: 18px;
0343: display: flex;
0344: align-items: center;
0345: justify-content: center;
0346: background: rgba(255, 255, 255, 0.18);
0347: }
0348:
0349: .feature-card__title {
0350: display: block;
0351: margin-top: 18px;
0352: font-size: 21px;
0353: font-weight: 800;
0354: }
0355:
0356: .feature-card__desc {
0357: display: block;
0358: margin-top: 8px;
0359: font-size: 14px;
0360: line-height: 1.65;
0361: color: rgba(255, 255, 255, 0.9);
0362: }
0363:
0364: .feature-card__meta {
0365: display: inline-flex;
0366: align-items: center;
0367: gap: 8px;
0368: min-height: 32px;
0369: padding: 0 12px;
0370: border-radius: 999px;
0371: background: rgba(255, 255, 255, 0.14);
0372: font-size: 13px;
0373: font-weight: 700;
0374: }
0375:
0376: .module-list-grid,
0377: .action-list,
0378: .status-grid,
0379: .record-list,
0380: .simple-list {
0381: display: flex;
0382: flex-direction: column;
0383: gap: 14px;
0384: }
0385:
0386: .module-card-pro {
0387: position: relative;
0388: overflow: hidden;
0389: border-radius: 24px;
0390: padding: 20px 18px;
0391: display: flex;
0392: align-items: center;
0393: gap: 16px;
0394: background: rgba(255, 255, 255, 0.95);
0395: border: 1px solid rgba(172, 199, 231, 0.46);
```

```
0396: box-shadow: 0 16px 32px rgba(19, 58, 107, 0.08);
0397: }
0398:
0399: .module-card-pro::after {
0400:   content: "";
0401:   position: absolute;
0402:   top: 16px;
0403:   right: 16px;
0404:   width: 62px;
0405:   height: 62px;
0406:   border-radius: 18px;
0407:   opacity: 0.2;
0408:   background:
0409:     radial-gradient(circle at center, currentColor 0 2px, transparent 2px 100%),
0410:     linear-gradient(currentColor, currentColor);
0411: }
0412:
0413: .module-card-pro--volunteer {
0414:   color: #0f8e88;
0415: }
0416:
0417: .module-card-pro--honor {
0418:   color: #cc8a11;
0419: }
0420:
0421: .module-card-pro__icon {
0422:   width: 54px;
0423:   height: 54px;
0424:   border-radius: 18px;
0425:   display: flex;
0426:   align-items: center;
0427:   justify-content: center;
0428:   flex-shrink: 0;
0429: }
0430:
0431: .module-card-pro--volunteer .module-card-pro__icon {
0432:   background: linear-gradient(135deg, rgba(29, 99, 216, 0.14), rgba(19, 171, 146, 0.18));
0433: }
0434:
0435: .module-card-pro--honor .module-card-pro__icon {
0436:   background: linear-gradient(135deg, rgba(29, 99, 216, 0.14), rgba(232, 176, 59, 0.18));
0437: }
0438:
0439: .module-card-pro__body {
0440:   flex: 1;
0441:   display: flex;
0442:   flex-direction: column;
0443:   gap: 8px;
0444: }
0445:
0446: .module-card-pro__title {
0447:   font-size: 19px;
0448:   font-weight: 800;
0449:   color: var(--app-text-primary);
0450: }
```

```
0451:
0452: .module-card-pro__desc {
0453:   font-size: 14px;
0454:   line-height: 1.7;
0455:   color: var(--app-text-secondary);
0456: }
0457:
0458: .module-card-pro__meta {
0459:   display: inline-flex;
0460:   align-items: center;
0461:   gap: 8px;
0462:   min-height: 30px;
0463:   padding: 0 12px;
0464:   border-radius: 999px;
0465:   background: var(--app-fill-soft);
0466:   color: var(--app-primary-deep);
0467:   font-size: 13px;
0468:   font-weight: 700;
0469: }
0470:
0471: .list-row-card {
0472:   display: flex;
0473:   align-items: flex-start;
0474:   justify-content: space-between;
0475:   gap: 14px;
0476:   padding: 18px;
0477:   border-radius: 22px;
0478:   background: rgba(255, 255, 255, 0.94);
0479:   border: 1px solid rgba(172, 199, 231, 0.46);
0480:   box-shadow: 0 16px 30px rgba(19, 58, 107, 0.08);
0481: }
0482:
0483: .list-row-card__body {
0484:   flex: 1;
0485: }
0486:
0487: .list-row-card__title {
0488:   display: block;
0489:   font-size: 18px;
0490:   font-weight: 800;
0491:   color: var(--app-text-primary);
0492:   line-height: 1.45;
0493: }
0494:
0495: .list-row-card__desc {
0496:   display: block;
0497:   margin-top: 8px;
0498:   font-size: 14px;
0499:   line-height: 1.7;
0500:   color: var(--app-text-secondary);
0501: }
0502:
0503: .list-row-card__meta {
0504:   display: inline-flex;
0505:   align-items: center;
```

```
0506: gap: 8px;
0507: margin-top: 12px;
0508: min-height: 30px;
0509: padding: 0 12px;
0510: border-radius: 999px;
0511: background: var(--app-fill-soft);
0512: color: var(--app-primary-deep);
0513: font-size: 13px;
0514: font-weight: 700;
0515: }
0516:
0517: .data-grid-two {
0518:   display: grid;
0519:   grid-template-columns: repeat(2, 1fr);
0520:   gap: 14px;
0521: }
0522:
0523: .data-stat-card {
0524:   border-radius: 24px;
0525:   padding: 18px 16px;
0526:   color: #ffffff;
0527:   box-shadow: 0 16px 30px rgba(19, 58, 107, 0.12);
0528: }
0529:
0530: .data-stat-card--blue {
0531:   background: linear-gradient(135deg, #1d63d8 0%, #3f88ef 100%);
0532: }
0533:
0534: .data-stat-card--green {
0535:   background: linear-gradient(135deg, #11907f 0%, #16b39f 100%);
0536: }
0537:
0538: .data-stat-card--amber {
0539:   background: linear-gradient(135deg, #c98713 0%, #efb547 100%);
0540: }
0541:
0542: .data-stat-card--red {
0543:   background: linear-gradient(135deg, #ca5b4d 0%, #e57d67 100%);
0544: }
0545:
0546: .data-stat-card__label {
0547:   display: block;
0548:   font-size: 13px;
0549:   color: rgba(255, 255, 255, 0.86);
0550: }
0551:
0552: .data-stat-card__value {
0553:   display: block;
0554:   margin-top: 14px;
0555:   font-size: 30px;
0556:   font-weight: 800;
0557:   line-height: 1;
0558: }
0559:
0560: .themed-form-card {
```

```
0561: padding: 18px;
0562: border-radius: 24px;
0563: background: rgba(255, 255, 255, 0.95);
0564: border: 1px solid rgba(172, 199, 231, 0.46);
0565: box-shadow: 0 18px 34px rgba(19, 58, 107, 0.08);
0566: }
0567:
0568: .field-shell {
0569:   display: flex;
0570:   flex-direction: column;
0571:   gap: 10px;
0572:   margin-bottom: 18px;
0573: }
0574:
0575: .field-label {
0576:   font-size: 16px;
0577:   font-weight: 800;
0578:   color: var(--app-text-primary);
0579: }
0580:
0581: .field-helper {
0582:   font-size: 13px;
0583:   line-height: 1.6;
0584:   color: var(--app-text-muted);
0585: }
0586:
0587: .picker-input-theme {
0588:   min-height: 52px;
0589:   padding: 0 16px;
0590:   border-radius: 16px;
0591:   background: #f9fbff;
0592:   border: 1px solid var(--app-border-soft);
0593:   display: flex;
0594:   align-items: center;
0595:   justify-content: space-between;
0596:   gap: 12px;
0597:   box-sizing: border-box;
0598: }
0599:
0600: .picker-input-theme text {
0601:   font-size: 16px;
0602:   color: var(--app-text-primary);
0603: }
0604:
0605: .picker-input-theme--muted text:first-child {
0606:   color: var(--app-text-light);
0607: }
0608:
0609: .toolbar-shell {
0610:   display: flex;
0611:   align-items: center;
0612:   justify-content: space-between;
0613:   gap: 12px;
0614:   padding: 16px 18px;
0615:   border-radius: 22px;
```



```
0616: background: rgba(255, 255, 255, 0.95);
0617: border: 1px solid rgba(172, 199, 231, 0.46);
0618: box-shadow: 0 16px 30px rgba(19, 58, 107, 0.08);
0619: }
0620:
0621: .toolbar-shell__text {
0622:   font-size: 14px;
0623:   color: var(--app-text-secondary);
0624: }
0625:
0626: .empty-state-pro {
0627:   display: flex;
0628:   flex-direction: column;
0629:   align-items: center;
0630:   justify-content: center;
0631:   gap: 12px;
0632:   min-height: 180px;
0633:   padding: 26px 22px;
0634:   text-align: center;
0635:   border-radius: 24px;
0636:   background: rgba(255, 255, 255, 0.94);
0637:   border: 1px solid rgba(172, 199, 231, 0.42);
0638:   box-shadow: 0 16px 32px rgba(19, 58, 107, 0.08);
0639: }
0640:
0641: .empty-state-pro__icon {
0642:   width: 68px;
0643:   height: 68px;
0644:   border-radius: 22px;
0645:   display: flex;
0646:   align-items: center;
0647:   justify-content: center;
0648:   background: linear-gradient(135deg, rgba(29, 99, 216, 0.14), rgba(22, 179, 159, 0.16));
0649: }
0650:
0651: .empty-state-pro__title {
0652:   font-size: 18px;
0653:   font-weight: 800;
0654:   color: var(--app-text-primary);
0655: }
0656:
0657: .empty-state-pro__desc {
0658:   font-size: 14px;
0659:   line-height: 1.7;
0660:   color: var(--app-text-secondary);
0661: }

/* 文件: miniprogram01/src/uni.promisify.adaptor.js */
0001: uni.addInterceptor({
0002:   returnValue(res) {
0003:     if (
0004:       !(
0005:         !!res &&
0006:         (typeof res === 'object' || typeof res === 'function') &&
0007:         typeof res.then === 'function'

```

```
0008:   )
0009: }{
0010:   return res
0011: }
0012: return new Promise((resolve, reject) => {
0013:   res.then((res) => {
0014:     if (!res) return resolve(res)
0015:     return res[0] ? reject(res[0]) : resolve(res[1])
0016:   })
0017: })
0018: }
0019: })

/* 文件: miniprogram01/src/uni.scss */
0001: /**
0002:  * 这里是uni-app内置的常用样式变量
0003:  *
0004:  * uni-app 官方扩展插件及插件市场 (https://ext.dcloud.net.cn) 上很多三方插件均使用了这些样式变量
0005:  * 如果你是插件开发者, 建议你使用scss预处理, 并在插件代码中直接使用这些变量 (无需 import 这个文件), 方便用户通过搭积木的方式开发整体风格一致的App
0006:  *
0007:  */
0008:
0009: @import "uview-plus/theme.scss";
0010:
0011: /**
0012:  * 如果你是App开发者 (插件使用者), 你可以通过修改这些变量来定制自己的插件主题, 实现自定义主题功能
0013:  *
0014:  * 如果你的项目同样使用了scss预处理, 你也可以直接在你的 scss 代码中使用如下变量, 同时无需 import 这个文件
0015:  */
0016:
0017: /* 颜色变量 */
0018:
0019: /* 行为相关颜色 */
0020: $uni-color-primary: #1d63d8;
0021: $uni-color-success: #0f9c7e;
0022: $uni-color-warning: #e3a63a;
0023: $uni-color-error: #c85b51;
0024:
0025: /* 文字基本颜色 */
0026: $uni-text-color: #12304e;//基本色
0027: $uni-text-color-inverse: #fff;//反色
0028: $uni-text-color-grey: #7f95a9;//辅助灰色, 如加载更多的提示信息
0029: $uni-text-color-placeholder: #8ba0b5;
0030: $uni-text-color-disable: #c0ceda;
0031:
0032: /* 背景颜色 */
0033: $uni-bg-color: #ffffff;
0034: $uni-bg-color-grey: #f5f9ff;
0035: $uni-bg-color-hover: #edf5ff;//点击状态颜色
0036: $uni-bg-color-mask: rgba(0, 0, 0, 0.4);//遮罩颜色
0037:
0038: /* 边框颜色 */
0039: $uni-border-color: #c6d7e8;
0040:
0041: /* 尺寸变量 */
```

```
0042:
0043: /* 文字尺寸 */
0044: $uni-font-size-sm:12px;
0045: $uni-font-size-base:16px;
0046: $uni-font-size-lg:16px;
0047:
0048: /* 图片尺寸 */
0049: $uni-img-size-sm:20px;
0050: $uni-img-size-base:26px;
0051: $uni-img-size-lg:40px;
0052:
0053: /* Border Radius */
0054: $uni-border-radius-sm: 2px;
0055: $uni-border-radius-base: 8px;
0056: $uni-border-radius-lg: 16px;
0057: $uni-border-radius-circle: 50%;
0058:
0059: /* 水平间距 */
0060: $uni-spacing-row-sm: 5px;
0061: $uni-spacing-row-base: 10px;
0062: $uni-spacing-row-lg: 15px;
0063:
0064: /* 垂直间距 */
0065: $uni-spacing-col-sm: 4px;
0066: $uni-spacing-col-base: 8px;
0067: $uni-spacing-col-lg: 12px;
0068:
0069: /* 透明度 */
0070: $uni-opacity-disabled: 0.3; // 组件禁用态的透明度
0071:
0072: /* 文章场景相关 */
0073: $uni-color-title: #12304e; // 文章标题颜色
0074: $uni-font-size-title:20px;
0075: $uni-color-subtitle: #35516f; // 二级标题颜色
0076: $uni-font-size-subtitle:26px;
0077: $uni-color-paragraph: #35516f; // 文章段落颜色
0078: $uni-font-size-paragraph:15px;

/* 文件: miniprogram01/src/utils/api-helper.js */
0001: import { unwrapApiData, resolveApiErrorMessage } from '@utils/api'
0002:
0003: export { unwrapApiData, resolveApiErrorMessage }

/* 文件: miniprogram01/src/utils/api.js */
0001: /** 统一解析后端响应, 失败时抛出带 message 的错误。 */
0002: export const unwrapApiData = (response, fallback = null) => {
0003:   const payload = response?.data
0004:   if (!payload || typeof payload !== 'object') {
0005:     throw new Error('接口返回格式异常')
0006:   }
0007:
0008:   if (Number(payload.code) !== 0) {
0009:     throw new Error(payload.message || '请求失败')
0010:   }
0011:
```

```
0012: return payload.data ?? fallback
0013: }
0014:
0015: /** 提取错误提示文案，避免页面重复写兜底逻辑。 */
0016: export const resolveApiErrorMessage = (error, fallback = '请求失败，请稍后重试') =>
0017: error?.message || error?.errMsg || fallback

/* 文件: miniprogram01/src/utls/auth.js */
0001: import { showErrorToast, showSuccessToast } from '@/utls/feedback'
0002:
0003: const LOGIN_PAGE = '/pages/login/login'
0004: let interceptorReady = false
0005:
0006: const getSubscribeTemplatIds = () => {
0007:   const raw = import.meta.env.VITE_SUBSCRIBE_TEMPLATE_IDS || ''
0008:   return raw
0009:     .split(',')
0010:     .map((item) => item.trim())
0011:     .filter(Boolean)
0012: }
0013:
0014: /** 统一获取当前页面路径，供全局守卫判断是否需要放行。 */
0015: const getCurrentRoute = () => {
0016:   const pages = getCurrentPages()
0017:   const currentPage = pages[pages.length - 1]
0018:   return currentPage ? `/${currentPage.route}` : ''
0019: }
0020:
0021: /** 判断当前跳转目标是否为登录页。 */
0022: const isLoginRoute = (url = '') => url.split('?')[0] === LOGIN_PAGE
0023:
0024: /** 在未完成合规前，统一跳回登录页。 */
0025: const redirectToLogin = (message = '') => {
0026:   if (message) {
0027:     showErrorToast(message)
0028:   }
0029:   if (!isLoginRoute(getCurrentRoute())) {
0030:     uni.reLaunch({ url: LOGIN_PAGE })
0031:   }
0032: }
0033:
0034: /** 校验隐私协议与用户认证状态，未通过时阻止继续访问业务页面。 */
0035: export const ensureComplianceReady = (userStore, options = {}) => {
0036:   const { redirect = true, toast = true } = options
0037:   let message = ''
0038:
0039:   if (!userStore.privacyAgreed) {
0040:     message = '请先同意隐私保护指引'
0041:   } else if (!userStore.hasRealname) {
0042:     message = '请先完成用户认证'
0043:   }
0044:
0045:   if (!message) {
0046:     return true
0047:   }
```

```
0048:
0049: if (redirect) {
0050:   redirectToLogin(toast ? message : '')
0051: } else if (toast) {
0052:   showErrorToast(message)
0053: }
0054:
0055: return false
0056: }
0057:
0058: /** 注册全局路由拦截，避免未完成认证的用户直接进入业务页。 */
0059: export const setupComplianceInterceptors = (userStore) => {
0060:   if (interceptorReady) return
0061:
0062:   const methods = ['navigateTo', 'redirectTo', 'reLaunch', 'switchTab']
0063:   methods.forEach((method) => {
0064:     uni.addInterceptor(method, {
0065:       invoke(args) {
0066:         const url = args?.url || ''
0067:         if (
0068:           !url ||
0069:           isLoginRoute(url) ||
0070:           ensureComplianceReady(userStore, { redirect: false, toast: false })
0071:         ) {
0072:           return args
0073:         }
0074:         setTimeout(() => redirectToLogin('请先完成隐私同意与用户认证'), 0)
0075:         return false
0076:       }
0077:     })
0078:   })
0079:   interceptorReady = true
0080: }
0081:
0082:
0083: /** 调用微信登录接口，获取前端会话 code。 */
0084: export const wxLogin = () =>
0085:   new Promise((resolve) => {
0086:     uni.login({
0087:       success: (res) => resolve(res),
0088:       fail: () => resolve(false)
0089:     })
0090:   })
0091:
0092: /** 在隐私同意后发起微信登录，并把会话 code 存入本地状态。 */
0093: export const completeLogin = async (userStore) => {
0094:   if (!userStore?.privacyAgreed) {
0095:     showErrorToast('请先同意隐私保护指引')
0096:     return false
0097:   }
0098:
0099:   uni.showLoading({ title: '登录中', mask: true })
0100:   const res = await wxLogin()
0101:   uni.hideLoading()
0102:
```

```
0103: if (!res?.code) {
0104:   uni.showModal({
0105:     title: '登录失败',
0106:     content: '暂时无法获取微信登录凭证，请稍后重试。',
0107:     showCancel: false
0108:   })
0109:   return false
0110: }
0111:
0112: userStore.setLoginSession({
0113:   sessionCode: res.code,
0114:   loginAt: new Date().toISOString()
0115: })
0116:
0117: return res
0118: }
0119:
0120: /** 规范化订阅消息状态，供页面提示与进度展示使用。 */
0121: const resolveSubscribeStatus = (result = {}, tmplIds = []) => {
0122:   const values = tmplIds.map((id) => result[id]).filter(Boolean)
0123:   if (values.includes('accept')) return 'accepted'
0124:   if (values.includes('ban')) return 'banned'
0125:   if (values.includes('filter')) return 'filtered'
0126:   if (values.includes('reject')) return 'rejected'
0127:   if (values.length === 0) return 'unknown'
0128:   return 'rejected'
0129: }
0130:
0131: /** 按订阅结果给用户明确反馈，避免点击后无感知。 */
0132: const showSubscribeResult = (status) => {
0133:   if (status === 'accepted') {
0134:     showSuccessToast('已开启审核结果通知')
0135:     return
0136:   }
0137:
0138:   if (status === 'banned') {
0139:     uni.showModal({
0140:       title: '订阅未开启',
0141:       content: '微信通知设置暂未允许接收订阅消息，请在微信设置中开启后再试。',
0142:       showCancel: false
0143:     })
0144:     return
0145:   }
0146:
0147:   if (status === 'filtered') {
0148:     showErrorToast('当前订阅消息暂不可申请')
0149:     return
0150:   }
0151:
0152:   if (status === 'rejected') {
0153:     showErrorToast('暂未订阅，可稍后再次申请')
0154:     return
0155:   }
0156:
0157:   showErrorToast('订阅状态未更新，请稍后重试')
```

```
0158: }
0159:
0160: /** 根据微信接口错误返回可理解的提示文案。 */
0161: const resolveSubscribeFailMessage = (error = {}) => {
0162:   const message = error?.errMsg || error?.message || ''
0163:
0164:   if (/can only be invoked by user TAP gesture/i.test(message)) {
0165:     return '请点击页面按钮发起订阅'
0166:   }
0167:
0168:   if (/not support|unsupported/i.test(message)) {
0169:     return '当前微信版本暂不支持订阅消息'
0170:   }
0171:
0172:   if (/cancel/i.test(message)) {
0173:     return '已取消订阅申请'
0174:   }
0175:
0176:   return '订阅申请未完成，请稍后重试'
0177: }
0178:
0179: /** 审核结果订阅申请，提交成功后调用。 */
0180: export const requestAuditSubscribeMessage = (userStore) => {
0181:   const tmplIds = getSubscribeTemplateIds()
0182:   if (tmplIds.length === 0) {
0183:     userStore?.setSubscribeStatus('unconfigured')
0184:     uni.showModal({
0185:       title: '订阅暂不可用',
0186:       content: '当前环境尚未配置审核结果通知模板，请联系管理员配置后再试。',
0187:       showCancel: false
0188:     })
0189:     return Promise.resolve(false)
0190:   }
0191:
0192:   if (typeof uni.requestSubscribeMessage !== 'function') {
0193:     userStore?.setSubscribeStatus('unsupported')
0194:     showErrorToast('当前环境暂不支持订阅消息')
0195:     return Promise.resolve(false)
0196:   }
0197:
0198:   return new Promise((resolve) => {
0199:     uni.requestSubscribeMessage({
0200:       tmplIds,
0201:       success: (res) => {
0202:         const status = resolveSubscribeStatus(res, tmplIds)
0203:         userStore?.setSubscribeStatus(status)
0204:         showSubscribeResult(status)
0205:         resolve(res)
0206:       },
0207:       fail: (error) => {
0208:         const failMessage = resolveSubscribeFailMessage(error)
0209:         userStore?.setSubscribeStatus(/cancel/i.test(error?.errMsg || '') ? 'cancelled' : 'failed')
0210:         showErrorToast(failMessage)
0211:         resolve(false)
0212:       }
0213:     })
0214:   })
0215: }
```

```
0213:  })
0214:  })
0215:  }
0216:
0217:  /** 将订阅状态转为页面可直接显示的中文文本。 */
0218:  export const getSubscribeStatusLabel = (status) => {
0219:    const mapping = {
0220:      unknown: '未申请',
0221:      accepted: '已订阅',
0222:      rejected: '已拒绝',
0223:      banned: '系统关闭',
0224:      filtered: '不可订阅',
0225:      cancelled: '已取消',
0226:      failed: '申请失败',
0227:      unsupported: '不支持',
0228:      unconfigured: '模板未配置'
0229:    }
0230:    return mapping[status] || '未申请'
0231:  }

/* 文件: miniprogram01/src/utls/feedback.js */
0001: /** 统一轻提示，保持页面反馈文案和展示方式一致。 */
0002: export const showInfoToast = (title) =>
0003:   uni.showToast({
0004:     title,
0005:     icon: 'none',
0006:     duration: 2200
0007:   })
0008:
0009: /** 统一错误提示。 */
0010: export const showErrorToast = (title) => showInfoToast(title)
0011:
0012: /** 统一成功提示。 */
0013: export const showSuccessToast = (title) => showInfoToast(title)

/* 文件: miniprogram01/src/utls/request.js */
0001: const BASE_URL = String(import.meta.env.VITE_BASE_URL || '').trim()
0002: const CLOUD_FUNCTION_NAME = String(
0003:   import.meta.env.VITE_CLOUD_FUNCTION_NAME || 'volunteer-service'
0004: ).trim()
0005: const CLOUD_ENV_ID = String(import.meta.env.VITE_CLOUD_ENV_ID || '').trim()
0006: const CLOUD_TIMEOUT_MS = Number(import.meta.env.VITE_CLOUD_TIMEOUT_MS || 12000)
0007:
0008: /** REST 路由与云函数 action 映射，确保前端统一按 URL 写法调用。 */
0009: const ROUTE_ACTION_MAP = {
0010:   'POST /auth/login': 'wechatLogin',
0011:   'POST /user/realname': 'bindUser',
0012:   'GET /user/profile': 'getUserProfile',
0013:   'POST /volunteer/submit': 'submitVolunteerDeclaration',
0014:   'GET /volunteer/records': 'getVolunteerRecords',
0015:   'POST /honor/submit': 'submitHonor',
0016:   'GET /honor/records': 'getHonorRecords',
0017:   'POST /admin/import': 'adminImport',
0018:   'GET /admin/dashboard': 'adminDashboardSummary',
0019:   'GET /admin/audit': 'adminAuditList',
```



```
0020: 'POST /admin/audit': 'adminAuditOperate',
0021: 'GET /admin/export': 'adminExport',
0022: 'GET /admin/users': 'adminGetUsers',
0023: 'POST /admin/users/role': 'adminSetUserRole',
0024: 'POST /admin/users/disable': 'adminDisableUser'
0025: }
0026:
0027: let wxCloudInitialized = false
0028:
0029: /** 从本地缓存读取 openid, 兼容独立缓存和 user_state 两种结构。 */
0030: const resolveOpenid = () => {
0031:   const directOpenid = uni.getStorageSync('yc_openid')
0032:   if (directOpenid) return String(directOpenid).trim()
0033:
0034:   const userState = uni.getStorageSync('yc_user_state')
0035:   if (userState && typeof userState === 'object' && userState.openid) {
0036:     return String(userState.openid).trim()
0037:   }
0038:
0039:   return ""
0040: }
0041:
0042: /** 在请求参数中自动注入 openid, 便于云函数识别当前用户。 */
0043: const attachOpenid = (data, openid) => {
0044:   if (!openid) return data
0045:
0046:   if (data == null) return { openid }
0047:
0048:   if (typeof data !== 'object' || Array.isArray(data)) return data
0049:
0050:   if (
0051:     Object.prototype.hasOwnProperty.call(data, 'openid') ||
0052:     Object.prototype.hasOwnProperty.call(data, '_openid')
0053:   ) {
0054:     return data
0055:   }
0056:
0057:   return { ...data, openid }
0058: }
0059:
0060: /** 统一规范化路由, 兼容 query、重复斜杠和尾斜杠。 */
0061: const normalizeRoutePath = (url = '') => {
0062:   const cleanPath = String(url || '')
0063:     .trim()
0064:     .split('?')[0]
0065:     .replace(/\\g, '/')
0066:     .replace(/\\{2,}/g, '/')
0067:
0068:   if (!cleanPath) return ""
0069:   const normalized = cleanPath.startsWith('/') ? cleanPath : `/${cleanPath}`
0070:   return normalized.endsWith('/') && normalized.length > 1 ? normalized.slice(0, -1) : normalized
0071: }
0072:
0073: /** 将 method 标准化为大写, 默认 GET。 */
0074: const normalizeMethod = (method = 'GET') =>
```

```
0075: String(method || 'GET')
0076:   .trim()
0077:   .toUpperCase()
0078:
0079: /** 判断字符串是否为完整 http(s) 地址。 */
0080: const isHttpUrl = (value = '') => /^https?:\:\/\/i.test(String(value || '').trim())
0081:
0082: /** BASE_URL 可用且为完整 URL 时走 HTTP 请求，否则走云函数。 */
0083: const shouldUseHttp = (url = '') => isHttpUrl(url) || (BASE_URL !== '' && isHttpUrl(BASE_URL))
0084:
0085: /** 计算 HTTP 最终请求地址，支持传入完整 URL 或相对路由。 */
0086: const resolveHttpUrl = (url = '') => {
0087:   if (isHttpUrl(url)) return String(url).trim()
0088:   return `${BASE_URL}${url || ''}`
0089: }
0090:
0091: /** 将路由 method+path 转为云函数 action。 */
0092: const resolveAction = (method, url) =>
0093:   ROUTE_ACTION_MAP[`${normalizeMethod(method)} ${normalizeRoutePath(url)}`] || ''
0094:
0095: /** 统一封装 uni.request。 */
0096: const requestByHttp = (options) =>
0097:   new Promise((resolve, reject) => {
0098:     uni.request({
0099:       ...options,
0100:       success: (res) => {
0101:         if (res.statusCode >= 200 && res.statusCode < 300) {
0102:           resolve(res)
0103:           return
0104:         }
0105:         reject(res)
0106:       },
0107:       fail: (error) => reject(error)
0108:     })
0109:   })
0110:
0111: /** 初始化微信云开发，仅在 mp-weixin 且存在 wx.cloud 时执行一次。 */
0112: const ensureWxCloudReady = () => {
0113:   if (wxCloudInitialized) return true
0114:   if (typeof wx === 'undefined' || !wx.cloud || typeof wx.cloud.init !== 'function') return false
0115:
0116:   try {
0117:     const initConfig = CLOUD_ENV_ID ? { env: CLOUD_ENV_ID, traceUser: true } : { traceUser: true }
0118:     wx.cloud.init(initConfig)
0119:     wxCloudInitialized = true
0120:     return true
0121:   } catch (error) {
0122:     console.warn('[request] wx.cloud.init failed:', error)
0123:     return false
0124:   }
0125: }
0126:
0127: /** 统一封装云函数调用，优先 wx.cloud，兜底 uniCloud。 */
0128: const requestByCloudFunction = ({ url, method, data, header, token }) =>
0129:   new Promise((resolve, reject) => {
```

```
0130: let settled = false
0131: const timeoutTimer = setTimeout(
0132:   () => {
0133:     if (settled) return
0134:     settled = true
0135:     reject({ errMsg: '请求超时，请稍后重试' })
0136:   },
0137:   Number.isFinite(CLOUD_TIMEOUT_MS) && CLOUD_TIMEOUT_MS > 0 ? CLOUD_TIMEOUT_MS : 12000
0138: )
0139:
0140: const action = resolveAction(method, url)
0141: const path = normalizeRoutePath(url)
0142:
0143: if (!action) {
0144:   clearTimeout(timeoutTimer)
0145:   settled = true
0146:   reject({ errMsg: `未匹配到云函数 action: ${normalizeMethod(method)} ${path}` })
0147:   return
0148: }
0149:
0150: const payload = {
0151:   action,
0152:   method: normalizeMethod(method),
0153:   path,
0154:   route: path,
0155:   data,
0156:   header: {
0157:     ...(header || {}),
0158:     ...(token ? { Authorization: `Bearer ${token}` } : {})
0159:   }
0160: }
0161:
0162: const handleSuccess = (res) => {
0163:   if (settled) return
0164:   settled = true
0165:   clearTimeout(timeoutTimer)
0166:
0167:   const result = res?.result
0168:   if (result && typeof result === 'object') {
0169:     resolve({ statusCode: 200, data: result })
0170:     return
0171:   }
0172:   resolve({
0173:     statusCode: 200,
0174:     data: {
0175:       code: 500,
0176:       message: '接口返回格式异常'
0177:     }
0178:   })
0179: }
0180:
0181: const handleFail = (error) => {
0182:   if (settled) return
0183:   settled = true
0184:   clearTimeout(timeoutTimer)
```

```
0185:   reject(error || { errMsg: '云函数调用失败' })
0186: }
0187:
0188: if (ensureWxCloudReady() && wx?.cloud?.callFunction) {
0189:   wx.cloud.callFunction({
0190:     name: CLOUD_FUNCTION_NAME,
0191:     data: payload,
0192:     success: handleSuccess,
0193:     fail: handleFail
0194:   })
0195:   return
0196: }
0197:
0198: if (typeof uniCloud !== 'undefined' && uniCloud?.callFunction) {
0199:   uniCloud.callFunction({
0200:     name: CLOUD_FUNCTION_NAME,
0201:     data: payload,
0202:     success: handleSuccess,
0203:     fail: handleFail
0204:   })
0205:   return
0206: }
0207:
0208: clearTimeout(timeoutTimer)
0209: settled = true
0210: reject({ errMsg: '当前环境未启用云函数能力，请检查云开发配置' })
0211: })
0212:
0213: /** 统一请求入口：有可用 BASE_URL 走 HTTP，否则走云函数。 */
0214: export const request = (options = {}) => {
0215:   const token = uni.getStorageSync('yc_token')
0216:   const openid = resolveOpenid()
0217:   const data = attachOpenid(options.data, openid)
0218:   const method = normalizeMethod(options.method || 'GET')
0219:   const header = {
0220:     ...(options.header || {}),
0221:     ...(token ? { Authorization: `Bearer ${token}` } : {})
0222:   }
0223:
0224:   if (shouldUseHttp(options.url)) {
0225:     return requestByHttp({
0226:       ...options,
0227:       method,
0228:       data,
0229:       header,
0230:       url: resolveHttpUrl(options.url || '')
0231:     })
0232:   }
0233:
0234:   return requestByCloudFunction({
0235:     url: options.url || '',
0236:     method,
0237:     data,
0238:     header,
0239:     token
```

```
0240: })
0241: }

/* 文件: miniprogram01/src/utils/rules.js */
0001: export const volunteerModules = [
0002: {
0003:   id: 'red-culture',
0004:   name: '传承红色文化（关心下一代）',
0005:   desc: '红色宣讲、青少年教育等',
0006:   min: 3,
0007:   max: 10,
0008:   rules: [
0009:     '开展党史、革命史、优良传统、老党员先进事迹等红色主题宣讲',
0010:     '参与青少年教育、辅导、关爱等志愿服务'
0011:   ]
0012: },
0013: {
0014:   id: 'community-governance',
0015:   name: '参与基层治理',
0016:   desc: '政策宣传、矛盾调解等',
0017:   min: 1,
0018:   max: 5,
0019:   rules: [
0020:     '参与社区政策宣传活动，传播政策知识',
0021:     '参与社区议事、政策建议、管理监督等工作',
0022:     '参与社区邻里矛盾、家庭纠纷调解工作',
0023:     '参与社区治安巡逻、信访维稳等维护稳定工作',
0024:     '参与社区健康宣传、义诊、环境整治等活动'
0025:   ]
0026: },
0027: {
0028:   id: 'enterprise-service',
0029:   name: '服务企业发展',
0030:   desc: '技术咨询、经验指导等',
0031:   min: 3,
0032:   max: 10,
0033:   rules: [
0034:     '以参加座谈会、调研、线下讲座、线上授课、课题研究等形式为企业提供技术咨询、经验指导、人才培养'
0035:   ]
0036: },
0037: {
0038:   id: 'elder-help',
0039:   name: '实施以老助老',
0040:   desc: '帮扶困难老同志等',
0041:   min: 1,
0042:   max: 5,
0043:   rules: ['帮扶困难老同志、独居老人等特殊群体']
0044: },
0045: {
0046:   id: 'other-service',
0047:   name: '其他服务',
0048:   desc: '未涵盖的社区服务',
0049:   min: 1,
0050:   max: 5,
0051:   rules: ['未涵盖的其他社区志愿服务']
}
```

```
0052: }
0053: ]
0054:
0055: export const honorLevels = [
0056: {
0057:   id: 'national',
0058:   name: '国家级荣誉',
0059:   points: 20
0060: },
0061: {
0062:   id: 'provincial',
0063:   name: '省部级荣誉',
0064:   points: 16
0065: },
0066: {
0067:   id: 'bureau',
0068:   name: '厅局级荣誉',
0069:   points: 12
0070: },
0071: {
0072:   id: 'factory',
0073:   name: '厂处级荣誉',
0074:   points: 10
0075: }
0076: ]
0077:
0078: export const getVolunteerModule = (moduleId) =>
0079: volunteerModules.find((item) => item.id === moduleId)
0080:
0081: export const getHonorLevel = (levelId) => honorLevels.find((item) => item.id === levelId)
0082:
0083: /** 脱敏展示姓名，避免在前端直接暴露完整身份信息。 */
0084: export const maskName = (value) => {
0085:   if (!value) return ''
0086:   if (value.length === 1) return `${value}*`
0087:   return `${value.slice(0, 1)}*`
0088: }
0089:
0090: export const maskIdCard = (value) => {
0091:   if (!value) return ''
0092:   if (value.length < 8) return value
0093:   return `${value.slice(0, 3)}*****${value.slice(-4)}`
0094: }
0095:
0096: /** 脱敏展示手机号，避免在前端完整暴露联系方式。 */
0097: export const maskPhone = (value) => {
0098:   if (!value) return ''
0099:   if (value.length < 7) return value
0100:   return `${value.slice(0, 3)}****${value.slice(-4)}`
0101: }
0102:
0103: export const isIdCardValid = (value) => {
0104:   if (!value) return false
0105:   const trimmed = value.trim()
0106:   return /^(^\d{15}$)(^\d{17}[\dXx]$)/.test(trimmed)
```

```
0107: }
0108:
0109: /** 校验中国大陆手机号格式。 */
0110: export const isPhoneValid = (value) => {
0111:   if (!value) return false
0112:   return /^1[3-9]\d{9}$/.test(value.trim())
0113: }

/* 文件: miniprogram01/src/utls/upload.js */
0001: import { showErrorToast } from '@/utls/feedback'
0002:
0003: const MAX_FILE_SIZE = 10 * 1024 * 1024
0004: const FILE_SELECT_UNSUPPORTED = 'unsupported'
0005:
0006: /** 上传前先说明用途，符合微信“先说明、后申请”的合规要求。 */
0007: export const confirmMediaPermission = (sourceType) =>
0008:   new Promise((resolve) => {
0009:     const sourceLabel = sourceType === 'camera' ? '相机' : '相册'
0010:     uni.showModal({
0011:       title: '权限申请说明',
0012:       content: `需要使用您的${sourceLabel}上传佐证材料，仅在您主动提交申报时使用。是否继续？`,
0013:       success: (res) => resolve(!!res.confirm),
0014:       fail: () => resolve(false)
0015:     })
0016:   })
0017:
0018: /** 打开系统权限设置页，便于用户重新授权。 */
0019: export const openSystemPermissionSetting = () =>
0020:   new Promise((resolve) => {
0021:     if (typeof uni.openSetting !== 'function') {
0022:       showErrorToast('当前环境暂不支持打开权限设置')
0023:       resolve(false)
0024:       return
0025:     }
0026:     uni.openSetting({
0027:       success: (res) => resolve(res),
0028:       fail: () => resolve(false)
0029:     })
0030:   })
0031: })
0032:
0033: /** 判断是否属于用户取消选择，取消不应被视为权限拒绝。 */
0034: const isCancelError = (message = '') => /cancel/i.test(message)
0035:
0036: /** 判断是否属于权限被拒绝，便于给出打开设置的引导。 */
0037: const isPermissionDeniedError = (message = '') =>
0038:   /auth deny|authorize|permission|denied/i.test(message)
0039:
0040: /** 统一调起图片选择，并过滤超过 50MB 的图片。 */
0041: export const chooseEvidencelImages = async ({
0042:   sourceType = 'album',
0043:   count = 1,
0044:   maxSize = MAX_FILE_SIZE
0045: } = {}) => {
0046:   const confirmed = await confirmMediaPermission(sourceType)
```

```
0047: if (!confirmed) {
0048:   return { files: [], status: 'cancelled' }
0049: }
0050:
0051: try {
0052:   const res = await new Promise((resolve, reject) => {
0053:     uni.chooseMedia({
0054:       count,
0055:       mediaType: ['image'],
0056:       sourceType: [sourceType],
0057:       sizeType: ['compressed'],
0058:       camera: sourceType === 'camera' ? 'back' : undefined,
0059:       success: resolve,
0060:       fail: reject
0061:     })
0062:   })
0063:
0064:   const selectedFiles = res?.tempFiles || []
0065:   const validFiles = selectedFiles
0066:     .filter((item) => Number(item.size || 0) <= maxSize)
0067:     .map((item, index) => ({
0068:       url: item.tempFilePath,
0069:       name: item.tempFilePath?.split('/').pop() || `evidence-${Date.now()}-${index + 1}.jpg`,
0070:       size: Number(item.size || 0),
0071:       type: 'image',
0072:       sourceType
0073:     }))
0074:
0075:   if (validFiles.length !== selectedFiles.length) {
0076:     showErrorToast('超过50MB的图片无法上传, 请重新选择')
0077:   }
0078:
0079:   return {
0080:     files: validFiles,
0081:     status: 'granted'
0082:   }
0083: } catch (error) {
0084:   const message = error?.errMsg || ''
0085:
0086:   if (isCancelError(message)) {
0087:     return { files: [], status: 'cancelled' }
0088:   }
0089:
0090:   if (isPermissionDeniedError(message)) {
0091:     uni.showModal({
0092:       title: '权限未开启',
0093:       content: '未获得所需权限, 是否前往系统设置重新开启? ',
0094:       success: async (res) => {
0095:         if (res.confirm) {
0096:           await openSystemPermissionSetting()
0097:         }
0098:       }
0099:     })
0100:     return { files: [], status: 'denied' }
0101:   }
```



```
0102:
0103:   showErrorToast('选择图片失败, 请重试')
0104:   return { files: [], status: 'failed' }
0105: }
0106: }
0107:
0108: /** 前端仅做流程占位, 真正的文本与图片安全校验由后端接入微信接口完成。 */
0109: export const checkContentSecurity = async (payload = {}) => {
0110:   const hasText = Object.values(payload?.text || {}).some((item) => `${item || ''}`.trim())
0111:   const hasFiles = Array.isArray(payload?.files) && payload.files.length > 0
0112:
0113:   if (!hasText && !hasFiles) {
0114:     return false
0115:   }
0116:
0117:   uni.showLoading({ title: '内容校验中', mask: true })
0118:   uni.hideLoading()
0119:   return true
0120: }
0121:
0122: /** 页面展示用的权限状态文案。 */
0123: export const getPermissionStatusLabel = (status) => {
0124:   const mapping = {
0125:     unknown: '未申请',
0126:     granted: '已允许',
0127:     denied: '已拒绝',
0128:     failed: '申请失败'
0129:   }
0130:   return mapping[status] || '未申请'
0131: }
0132:
0133: /** 支持上传的文件扩展名, 传给微信接口时不带点号。 */
0134: const ALLOWED_FILE_EXTENSIONS = ['doc', 'docx', 'xls', 'xlsx', 'ppt', 'pptx', 'pdf', 'txt', 'csv']
0135:
0136: /** 上传文件前的权限说明 */
0137: export const confirmFilePermission = () =>
0138:   new Promise((resolve) => {
0139:     uni.showModal({
0140:       title: '权限申请说明',
0141:       content:
0142:         '需要访问您的文件以上传佐证材料 (如 Word、Excel、PDF 等), 仅在您主动提交申报时使用。是否继续?',
0143:       success: (res) => resolve(!!res.confirm),
0144:       fail: () => resolve(false)
0145:     })
0146:   })
0147:
0148: /** 获取文件路径, 兼容 chooseMessageFile 与 chooseFile 返回结构。 */
0149: const getSelectedFilePath = (item = {}) => item.tempFilePath || item.path || item.url || ''
0150:
0151: /** 获取文件名, 优先使用微信返回的 name, 兜底从路径解析。 */
0152: const getSelectedFileName = (item = {}, index = 0) => {
0153:   const filePath = getSelectedFilePath(item)
0154:   return item.name || filePath.split('/').pop() || `file-${Date.now()}-${index + 1}`
0155: }
0156:
```

```
0157: /** 判断用户所选文件扩展名是否在允许范围内。 */
0158: const isAllowedEvidenceFile = (item = {}, index = 0) => {
0159:   const fileName = getSelectedFileName(item, index)
0160:   const extension = fileName.split('.').pop()?.toLowerCase() || ''
0161:   return ALLOWED_FILE_EXTENSIONS.includes(extension)
0162: }
0163:
0164: /** 使用微信消息文件选择器选取文件，小程序端优先使用该能力。 */
0165: const chooseMessageEvidenceFiles = ({ count }) =>
0166:   new Promise((resolve, reject) => {
0167:     if (typeof uni.chooseMessageFile !== 'function') {
0168:       reject(new Error(FILE_SELECT_UNsupported))
0169:       return
0170:     }
0171:
0172:     uni.chooseMessageFile({
0173:       count,
0174:       type: 'file',
0175:       extension: ALLOWED_FILE_EXTENSIONS,
0176:       success: resolve,
0177:       fail: reject
0178:     })
0179:   })
0180:
0181: /** 使用通用文件选择器兜底，兼容部分非微信端或旧构建环境。 */
0182: const chooseSystemEvidenceFiles = ({ count }) =>
0183:   new Promise((resolve, reject) => {
0184:     if (typeof uni.chooseFile !== 'function') {
0185:       reject(new Error(FILE_SELECT_UNsupported))
0186:       return
0187:     }
0188:
0189:     uni.chooseFile({
0190:       count,
0191:       type: 'file',
0192:       extension: ALLOWED_FILE_EXTENSIONS,
0193:       success: resolve,
0194:       fail: reject
0195:     })
0196:   })
0197:
0198: /** 选择原始文件结果，微信端优先走 chooseMessageFile，非取消失败时再兜底。 */
0199: const chooseRawEvidenceFiles = async ({ count }) => {
0200:   try {
0201:     return await chooseMessageEvidenceFiles({ count })
0202:   } catch (messageFileError) {
0203:     const message = messageFileError?.errMsg || messageFileError?.message || ''
0204:     if (isCancelError(message)) {
0205:       throw messageFileError
0206:     }
0207:   }
0208:
0209:   return chooseSystemEvidenceFiles({ count })
0210: }
0211:
```

```
0212: /** 统一调起文件选择，支持 Word、Excel、PDF 等常见格式。 */
0213: export const chooseEvidenceFiles = async ({ count = 1, maxSize = MAX_FILE_SIZE } = {}) => {
0214:   const confirmed = await confirmFilePermission()
0215:   if (!confirmed) {
0216:     return { files: [], status: 'cancelled' }
0217:   }
0218:
0219:   try {
0220:     const res = await chooseRawEvidenceFiles({ count })
0221:     const selectedFiles = res?.tempFiles || []
0222:     const allowedFiles = selectedFiles.filter(isAllowedEvidenceFile)
0223:     const validFiles = allowedFiles
0224:       .filter((item) => Number(item.size || 0) <= maxSize)
0225:       .map((item, index) => ({
0226:         url: getSelectedFilePath(item),
0227:         name: getSelectedFileName(item, index),
0228:         size: Number(item.size || 0),
0229:         type: 'file',
0230:         mimeType: item.type || ''
0231:       }))
0232:
0233:     if (allowedFiles.length !== selectedFiles.length) {
0234:       showErrorToast('仅支持 Word、Excel、PPT、PDF、TXT、CSV 文件')
0235:     }
0236:
0237:     if (validFiles.length !== allowedFiles.length) {
0238:       showErrorToast('部分文件超过 10MB，已自动过滤')
0239:     }
0240:
0241:     return {
0242:       files: validFiles,
0243:       status: 'granted'
0244:     }
0245:   } catch (error) {
0246:     const message = error?.errMsg || error?.message || ''
0247:
0248:     if (isCancelError(message)) {
0249:       return { files: [], status: 'cancelled' }
0250:     }
0251:
0252:     if (message === FILE_SELECT_UNSUPPORTED) {
0253:       showErrorToast('当前环境暂不支持选择文件，请改用图片上传')
0254:       return { files: [], status: 'unsupported' }
0255:     }
0256:
0257:     if (isPermissionDeniedError(message)) {
0258:       uni.showModal({
0259:         title: '权限未开启',
0260:         content: '未获得所需权限，是否前往系统设置重新开启？',
0261:         success: async (res) => {
0262:           if (res.confirm) {
0263:             await openSystemPermissionSetting()
0264:           }
0265:         }
0266:       })
0267:     }
0268:   }
0269: }
```

```
0267:   return { files: [], status: 'denied' }
0268: }
0269:
0270:   showErrorToast('选择文件失败，请重试')
0271:   return { files: [], status: 'failed' }
0272: }
0273: }

/* 文件: miniprogram01/vite.config.js */
0001: import { defineConfig } from 'vite'
0002: import path from 'path'
0003: import uni from '@dcloudio/vite-plugin-uni'
0004:
0005: export default defineConfig({
0006:   plugins: [uni()],
0007:   css: {
0008:     preprocessorOptions: {
0009:       scss: {
0010:         quietDeps: true,
0011:         silenceDeprecations: ['legacy-js-api', 'import']
0012:       }
0013:     }
0014:   },
0015:   resolve: {
0016:     alias: {
0017:       '@': path.resolve(__dirname, 'src')
0018:     }
0019:   }
0020: })
```